

Trường Đại học Văn Lang
KHOA CÔNG NGHỆ THÔNG TIN

“NHẬP MÔN PHÂN TÍCH DỮ LIỆU LỚN”
71ITDS40303

Chuyên ngành Công nghệ Dữ liệu
Bộ môn Khoa học Dữ liệu

Năm học 2022 – 2023 (Khóa 25)
TP. HCM - 2023

Mục lục

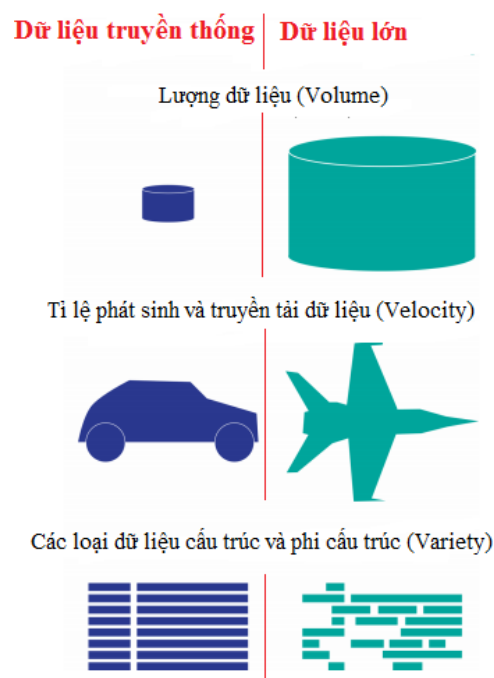
Chương 1. Giới thiệu về dữ liệu lớn.....	4
1.1. Định nghĩa dữ liệu lớn.....	4
1.2 Nguồn gốc của dữ liệu lớn.....	5
1.3 Đặc trưng của dữ liệu lớn.....	5
1.4 Sự khác nhau giữa dữ liệu lớn và dữ liệu truyền thống.....	7
1.5 Các công nghệ chính trong phân tích dữ liệu lớn.....	7
1.6. Mô hình xử lý dữ liệu lớn.....	8
1.7 Giải pháp phân tích dữ liệu lớn HDInsight của Microsoft.....	10
1.8 Giải pháp dữ liệu lớn MapReduce của Google.....	10
1.8.1 Quá trình Split.....	11
1.8.2 Quá trình Map và Shuffle.....	11
1.8.3 Quá trình Reduce.....	11
1.8.4 Một số thí dụ ứng dụng mô hình MapReduce.....	11
1.9 Giải pháp dữ liệu lớn Data Lake của Amazon.....	12
Chương 2. Cơ sở dữ liệu MongoDB.....	14
2.1 Cơ sở dữ liệu NoSQL.....	14
2.2 Các tính năng của MongoDB.....	14
2.3 Ví dụ về MongoDB.....	14
2.4 Các thành phần chính của kiến trúc MongoDB.....	15
2.5 Tại sao sử dụng MongoDB?.....	16
2.6 Mô hình hóa dữ liệu trong MongoDB.....	17
2.7 Sự khác biệt giữa MongoDB và RDBMS.....	18
2.8 Hướng dẫn các loại cơ sở dữ liệu NoSQL.....	19
2.8.1 NoSQL là gì?.....	19
2.8.2 Tại sao NoSQL?.....	20
2.8.3 Các tính năng của NoSQL.....	21
2.8.4 API đơn giản.....	22
2.8.5 Phân phối.....	22
2.8.6 Các loại cơ sở dữ liệu NoSQL.....	23
2.8.7 Công cụ cơ chế truy vấn cho NoSQL.....	26
2.9 Ưu điểm của NoSQL.....	26
2.10 Nhược điểm của NoSQL.....	27
2.11 Tóm lược.....	27
Chương 3 Cụm phân tích dữ liệu lớn Hadoop.....	29
3.1 Giới thiệu.....	29
3.2 MapReduce Layer.....	29
3.3 Hadoop Distributed File System (HDFS).....	30
3.3.1 Định nghĩa của HDFS.....	30
3.3.2 Đặc điểm của HDFS.....	30
3.3.3 Các khái niệm chính của HDFS.....	31
3.4 Các thành phần của HDFS.....	33
3.4.1 DataNode.....	34
3.4.2. NameNode.....	34
3.4.3 Secondary NameNode.....	35
3.4.4 MapReduce.....	36
3.4.5. Yarn.....	36

3.4.6 Hive.....	37
3.4.7 Pig.....	38
3.4.8. Hbase.....	38
Chương 4 Thực hành Phân tích Dữ liệu lớn Hadoop.....	40
4.1 Môi trường Hadoop phục vụ phân tích dữ liệu lớn.....	40
4.1.1 Cài đặt phần mềm MS Visual Studio Code (VSC).....	40
4.1.2. Cài đặt tiện ích SSH Remote Connect Extention trong VSC.....	40
4.1.3. Kiểm tra tài khoản lưu trữ dữ liệu cá nhân và thay đổi mật khẩu.....	40
4.1.4. Duyệt thư mục lưu trữ dữ liệu cá nhân (Remote Explorer).....	40
4.1.5. Truy cập thư mục lưu trữ dữ liệu cá nhân trong điện toán đám mây.....	40
4.2 Bài toán tính số lần lặp của một từ trong một tập tin văn bản.....	40
Đầu vào (dữ liệu tập tin văn bản).....	40
Đầu ra của tính toán (kết quả phân tích dữ liệu).....	41
Công cụ tính toán.....	41
4.2.1 Phân tích dữ liệu văn bản <mapper.py>.....	41
4.2.2 – Tổng hợp dữ liệu văn bản <reducer.py>.....	42
4.2.3 Kiểm tra và chạy các mã Python trong mapper.py và reducer.py.....	43
4.2.4 Tạo lập dữ liệu HDFS và khởi động Môi trường Hadoop.....	44
4.2.5 Chép dữ liệu nguồn (tập tin văn bản text) vào hệ thống tập tin HDFS.....	44
4.2.6 Chạy các lệnh mapper.py và reducer.py trên Hadoop (Map/Reduce).....	45
4.2.7 Kiểm tra kết quả phân tích dữ liệu trên Hadoop MapReduce.....	45
4.2.8 Các biến môi trường sử dụng để vận hành Hadoop Cluster.....	45
4.2.9 Các tập tin cấp hình Hadoop.....	46
4.3 Cải tiến mã Python sử dụng cặp (key,value) để chạy với Hadoop Cluster.....	47
Chương 5 Hướng dẫn viết đồ án môn học.....	50
5.1 Bài toán tìm kiếm bạn bè trên Facebook bằng công cụ Hadoop.....	50
5.2 Bài toán phân tích log files từ phần mềm Elearning (Moodle).....	52
5.3 Các chủ đề ứng dụng phân tích và trực quan hóa dữ liệu bằng MongoDB.....	52
Tài liệu tham khảo.....	55

Chương 1. Giới thiệu về dữ liệu lớn

1.1. Định nghĩa dữ liệu lớn

Mỗi phút người dùng mạng Internet gửi 204 triệu email, tạo ra 1,8 triệu lượt thích trên Facebook, gửi 278 nghìn Tweet và tải 200.000 ảnh lên Facebook. Mỗi ngày, con người tạo ra khoảng 2.5×10^{18} byte dữ liệu. Khoảng 90% dữ liệu trên thế giới ngày nay được tạo ra chỉ trong 2 năm vừa qua. Sự tăng tốc trong việc sản sinh ra thông tin đã tạo ra một nhu cầu cần có các công nghệ mới. *Phân tích dữ liệu lớn* (“*BigData Analytics*”) ra đời để thực hiện các công việc phân tích dữ liệu trên *phạm vi rộng lớn* vượt quá khả năng của các công nghệ xử lý dữ liệu truyền thống hiện nay.



Hình 1. Những khác biệt của dữ liệu lớn với dữ liệu truyền thống

Sự xuất hiện của *Phân tích dữ liệu lớn* giúp mở rộng quy mô của các hệ thống quản lý dữ liệu, sử dụng tập hợp các nguồn tài nguyên phân tán với các bộ vi xử lý nhanh hơn, lưu trữ được nhiều dữ liệu hơn. Công nghệ mới giúp chúng ta tận dụng được tất cả các nguồn dữ liệu sẵn có để cung cấp các phân tích dữ liệu tốt hơn và nhanh hơn trong nhiều lĩnh vực ứng dụng.

Trích dẫn định nghĩa Dữ liệu lớn

"Big Data are high-volume, high-velocity, and/or high-variety information assets that require new forms of processing to enable enhanced decision making, insight discovery and process optimization" -- (Gartner 2012)

“Dữ liệu lớn” là những tài sản thông tin có dung lượng lớn, phát triển nhanh chóng dưới nhiều hình thức khác nhau, đòi hỏi phải có hình thức xử lý mới để đưa ra quyết định, khám phá và tối ưu hóa quy trình.

1.2 Nguồn gốc của dữ liệu lớn

Dữ liệu lớn được hình thành chủ yếu từ các nguồn:

- (1) Dữ liệu *hành chính* (phát sinh từ hoạt động của tổ chức, có thể là chính phủ hay phi chính phủ). Ví dụ, hồ sơ y tế điện tử ở bệnh viện, hồ sơ học tập của học sinh, sinh viên, hồ sơ bảo hiểm, ngân hàng v.v.;
- (2) Dữ liệu từ hoạt động *thương mại* (phát sinh từ các giao dịch giữa hai thực thể). Ví dụ, các giao dịch thẻ tín dụng, giao dịch trực tuyến (trên mạng, bao gồm cả từ các thiết bị di động);
- (3) Dữ liệu từ các thiết bị *cảm biến* như hình ảnh vệ tinh, cảm biến thời tiết, cảm biến khí hậu;
- (4) Dữ liệu từ các thiết bị *giám sát*, ví dụ dữ liệu từ điện thoại di động, GPS;
- (5) Dữ liệu từ các *hành vi*, ví dụ tìm kiếm trực tuyến về một sản phẩm, một dịch vụ hay bất kỳ loại thông tin khác, trang thông tin trực tuyến;
- (6) Dữ liệu từ các thông tin trao đổi trên các phương tiện truyền thông xã hội.

1.3 Đặc trưng của dữ liệu lớn

Dữ liệu lớn có **03** đặc trưng cơ bản (mô hình **3Vs** về dữ liệu lớn):

- **Dung lượng (Volume):** Đây là đặc điểm tiêu biểu nhất của dữ liệu lớn (dung lượng dữ liệu rất lớn). Kích cỡ của Dữ liệu lớn đang từng ngày tăng lên, nó có thể nằm trong khoảng vài chục ngàn Terabyte (TB) cho đến nhiều ngàn petabyte (PB) (1 Petabyte = 1024 Terabyte) chỉ với một tập hợp dữ liệu. Dữ liệu truyền thống có thể lưu trữ trên các thiết bị như ổ đĩa cứng, thiết bị SAN/NAS... nhưng với dữ liệu lớn, chỉ có công nghệ “*đám mây*” phân tán mới có khả năng lưu trữ được.

- **Tốc độ (Velocity):** Tốc độ dữ liệu có thể hiểu theo 2 khía cạnh:

+ Sự gia tăng của dung lượng dữ liệu: ví dụ mỗi giây có tới hàng trăm triệu các yêu cầu truy cập tìm kiếm trên trang bán hàng của hãng Amazon;

+ Nhu cầu xử lý dữ liệu ở mức thời gian thực (real-time), có nghĩa dữ liệu cần được xử lý ngay tức thời ngay sau khi chúng phát sinh (tính đến milisecond).

Các ứng dụng phổ biến trên Internet như tài chính, ngân hàng, hàng không, quân sự, y tế – chăm sóc sức khỏe như hiện nay phần lớn là dữ liệu lớn, được xử lý ở mức thời gian thực. Công nghệ xử lý dữ liệu lớn ngày một tiên tiến, cho phép có thể xử lý ngay tức thì trước khi chúng được lưu trữ vào cơ sở dữ liệu.

- **Đa dạng (Variety):** Nếu như với dữ liệu truyền thống thường là dữ liệu có cấu trúc, thì ngày nay có hơn 90% dữ liệu phát sinh là phi cấu trúc (tài liệu, blog, hình ảnh, vi deo, bài hát, dữ liệu từ thiết bị cảm biến vật lý, thiết bị chăm sóc sức khỏe...). Công nghệ dữ liệu lớn cho phép liên kết và phân tích nhiều dạng dữ liệu khác nhau, như các comments/post của một nhóm người dùng nào đó trên Facebook, thông tin video được chia sẻ từ Youtube / Twitter, v.v...

Với nhu cầu và sự phát triển của các công nghệ phân tích dữ liệu lớn, người ta còn bổ xung thêm 02 đặc trưng quan trọng nữa của dữ liệu lớn (Mô hình **5Vs**) là:

- **Độ tin cậy / độ chính xác (Veracity):** Một trong những tính chất phức tạp nhất của dữ liệu lớn là độ tin cậy/ độ chính xác của dữ liệu. Với xu hướng phát triển của các phương tiện truyền thông xã hội (Social Media), mạng xã hội (Social Network) và sự gia tăng mạnh mẽ tính *tương tác* và *chia sẻ* của người dùng thiết bị di động đã làm cho bức tranh xác định về độ tin cậy và độ chính xác của dữ liệu trở nên ngày một khó khăn hơn. Bài toán *phân tích* và *loại bỏ dữ liệu* thiếu chính xác, nhiễu đang trở thành một đặc tính quan trọng của dữ liệu lớn.

- **Giá trị (Value):** Giá trị là đặc điểm quan trọng nhất của dữ liệu lớn, vì khi bắt đầu triển khai xây dựng các kho lưu trữ và xử lý dữ liệu lớn, việc đầu tiên chúng ta cần phải làm đó là xác định được giá trị của thông tin mang lại. Khi đó chúng ta mới có thể quyết định nên triển khai dữ liệu lớn hay không. Nếu chúng ta có dữ liệu lớn mà chỉ nhận được 1% lợi ích từ nó, thì không nên đầu tư vào dữ liệu lớn. Kết quả *dự báo chính xác* thể hiện rõ nét nhất về *giá trị* mà dữ liệu lớn có thể mang lại. Ví dụ, từ khối dữ liệu phát sinh trong quá trình khám, chữa bệnh sẽ giúp dự báo về sức khỏe được chính xác hơn, sẽ làm giảm được chi phí điều trị và các chi phí liên quan đến y tế – chăm sóc sức khỏe người dân.

1.4 Sự khác nhau giữa dữ liệu lớn và dữ liệu truyền thống

Dữ liệu lớn khác với dữ liệu truyền thống (thường lưu trữ trong các kho dữ liệu (data warehouse) ở các điểm:

- **Dữ liệu đa dạng hơn:** Khi khai thác dữ liệu truyền thống (thường là dữ liệu có cấu trúc), chúng ta thường phải trả lời các câu hỏi: Dữ liệu lấy ra kiểu gì? định dạng dữ liệu như thế nào? Đối với dữ liệu lớn, không phải trả lời các câu hỏi trên. Hay nói khác, khi khai thác, phân tích dữ liệu lớn chúng ta không cần quan tâm đến kiểu dữ liệu và định dạng của chúng; điều quan tâm là giá trị mà dữ liệu mang lại có đáp ứng được cho công việc hiện tại và tương lai hay không.

- **Lưu trữ dữ liệu lớn hơn:** Lưu trữ dữ liệu truyền thống vô cùng phức tạp và luôn đặt ra câu hỏi lưu thế nào: dung lượng kho lưu trữ bao nhiêu là đủ? Gắn kèm với câu hỏi đó là chi phí đầu tư (thường rất lớn). Công nghệ lưu trữ dữ liệu lớn hiện nay đã có thể giải quyết được vấn đề trên nhờ vào lưu trữ đám mây, lưu trữ phân tán, kết nối dữ liệu phân tán với nhau một cách chính xác, tốc độ xử lý nhanh hơn.

- **Truy vấn dữ liệu nhanh hơn:** dữ liệu lớn được cập nhật liên tục, trong khi các kho dữ liệu truyền thống thì ít được cập nhật và theo dõi thường xuyên, do đó thường gây ra tình trạng lỗi cấu trúc, lỗi truy vấn và dẫn đến tình trạng không tìm kiếm được thông tin đáp ứng theo yêu cầu.

- **Độ chính xác cao hơn:** dữ liệu lớn khi đưa vào sử dụng thường được kiểm định lại với những điều kiện chặt chẽ, số lượng thông tin cần được kiểm chứng thường rất lớn, để đảm bảo nguồn dữ liệu không chịu sự tác động nào của con người (nhập liệu, thay đổi số liệu thu thập được).

Nhiều nghiên cứu tìm hiểu về ứng dụng của phân tích dữ liệu lớn và các lĩnh vực trong đó phân tích dữ liệu lớn có thể được áp dụng đã chỉ ra **bốn lợi ích lớn** mà phân tích dữ liệu lớn có thể mang lại đó là: **cắt giảm chi phí, cắt giảm thời gian cho xử lý dữ liệu, tăng cơ hội phát triển, tối ưu hóa sản phẩm**, đồng thời **hỗ trợ đưa ra các quyết định đúng đắn và hợp lý hơn**

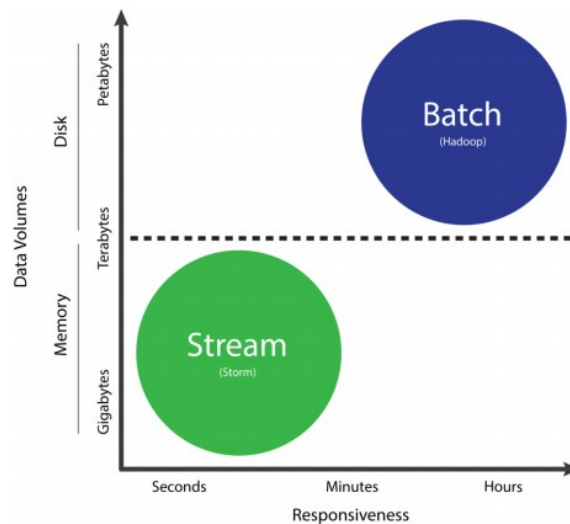
1.5 Các công nghệ chính trong phân tích dữ liệu lớn

Các công nghệ sử dụng trong phân tích dữ liệu lớn có thể được chia thành hai nhóm: **xử lý theo lô** (Batch Processing) và **xử lý theo luồng** (Stream Processing).

Xử lý theo lô là giải thuật dùng để xử lý những dữ liệu có dung lượng lớn.

Dữ liệu sẽ được thu thập, lưu trữ và được xử lý hàng loạt. **Hadoop** là một trong những công nghệ phổ biến nhất cho xử lý dữ liệu lớn theo lô. Nền tảng Hadoop cung cấp cho các nhà phát triển các thành phần chính như **hệ thống tập tin phân tán Hadoop** (Hadoop Distributed File System - *HDFS*) và **mô hình lập trình MapReduce**, cho phép xử lý đồng thời dữ liệu phân tán và song song, giúp giải quyết các vấn đề thường xuyên xảy ra trong xử lý dữ liệu với quy mô lớn.

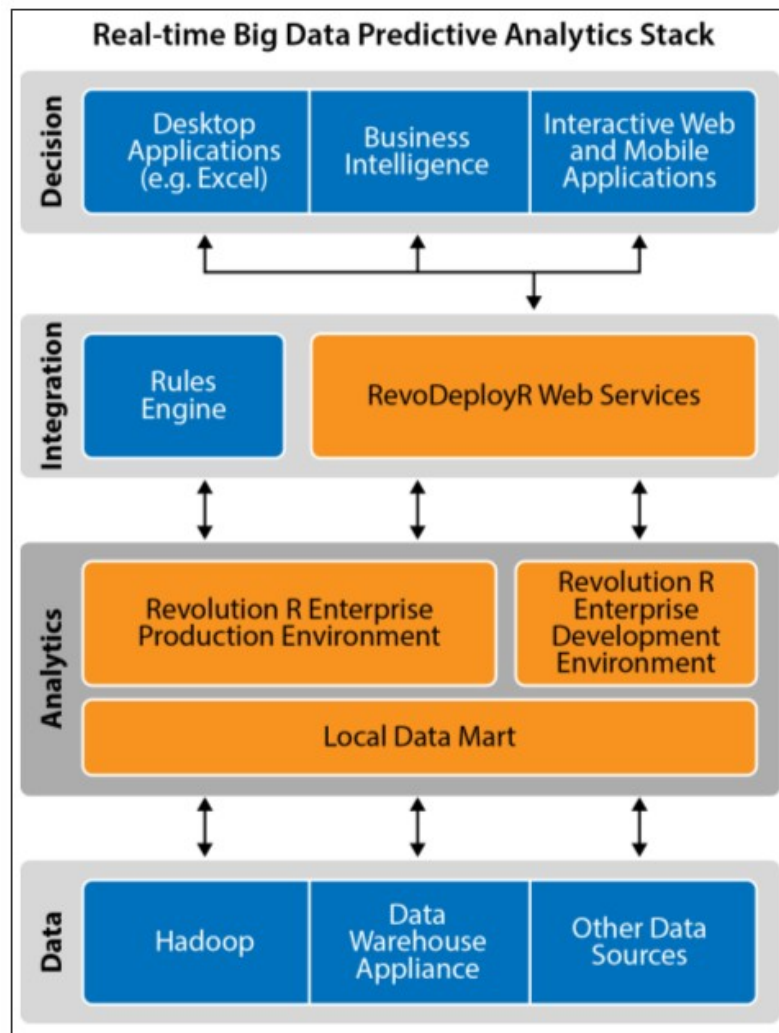
Xử lý theo luồng chú trọng đến *tốc độ xử lý* (thời gian thực) của dữ liệu, khi dữ liệu được phát sinh và truyền tải liên tục, cần được xử lý trong khoảng thời gian thực (real-time). Hiện tại chưa có công nghệ chủ đạo nào cho bài toán xử lý dữ liệu theo luồng, nhưng đây là một lĩnh vực đang được nghiên cứu và phát triển mạnh mẽ.



Hình 2. Xử lý theo lô và xử lý theo luồng

1.6. Mô hình xử lý dữ liệu lớn

Kiến trúc chung nhất cho xử lý dữ liệu lớn là sự tập hợp của các chương trình và công cụ để ra được **mô hình hoạt động** với sự đồng thuận cao nhất từ người sử dụng, nhà cung cấp, các nhà đầu tư, các nhà điều hành, quản lý.



Hình 6: Mô hình 4 lớp cho RTBDA[7]

- **Lớp dữ liệu:** Gồm có dữ liệu có cấu trúc trong RDBMS, NoSQL, HBase hay Impala; dữ liệu không cấu trúc trong Hadoop MapReduce; dữ liệu dòng đến từ hệ thống web, mạng xã hội, các cảm biến, hệ điều hành. Các bộ công cụ Hive, HBase, Storm và Spark cũng sẽ nằm ở lớp này.

- **Lớp phân tích:** nằm ở phía trên lớp dữ liệu. Lớp phân tích bao gồm một môi trường để phân tích thời gian thực, 1 môi trường phát triển các mô hình và một kho dữ liệu nội bộ được cập nhật theo chu kỳ từ lớp dữ liệu. Kho dữ liệu nội bộ này nằm gần các engine phân tích dữ liệu để có thể cải thiện tốc độ xử lý, dữ liệu được làm sạch từ lớp dữ liệu

- **Lớp tích hợp:** Là lớp nằm trên lớp phân tích. Nó làm nhiệm vụ gắn kết từ phía ứng dụng người dùng và engine phân tích. Lớp này thường bao gồm một cơ cấu luật hoặc cơ cấu xử lý sự kiện phức tạp, và một API dành cho các phân tích động (là cơ chế kết nối giữa các người phát triển ứng dụng và nhà khoa học dữ liệu)

- **Lớp ra quyết định:** là lớp trên cùng. Đây là nơi dành cho các ứng dụng người dùng cuối: như ứng dụng để bàn, ứng dụng mobile, các ứng dụng web tương tác. Đây là lớp mà hầu hết người dùng cuối nhìn thấy và thực hiện tương tác với hệ thống phân tích dữ liệu thời gian thực

Điều quan trọng cần chú ý là mỗi lớp sẽ liên kết với từng tập người dùng khác nhau. Mỗi lớp chủ động sử dụng công nghệ khác nhau, và có những giới hạn phạm vi trong việc triển khai phân tích dữ liệu lớn.

1.7 Giải pháp phân tích dữ liệu lớn HDInsight của Microsoft

Giải pháp Bigdata của Microsoft dựa trên nền tảng máy chủ SQL Server, Hadoop, Windows Azure và Windows Server, cung cấp các công cụ quản lý, mở rộng nhằm đạt được cái nhìn sâu sắc hơn về dữ liệu của doanh nghiệp, thúc đẩy hiệu quả kinh doanh.

Microsoft Bigdata cho phép quản lý hầu như bất kỳ loại dữ liệu nào, bất kể kích thước hoặc vị trí. Microsoft sử dụng SQL server 2012 và SQL server Parallel Data Warehouse để quản lý các dữ liệu lớn có cấu trúc. Với dữ liệu phi cấu trúc, Microsoft sử dụng Hadoop trên Windows Azure và Windows Server, sẽ cho phép xử lý dữ liệu phi cấu trúc với quy mô hàng Petabyte.

Với dữ liệu cần được xử lý theo luồng, Microsoft thường sử dụng công cụ SQL Server StreamInsight để quản lý các dữ liệu luồng trong thời gian thực.

Nhu cầu về xử lý dữ liệu lớn thời gian thực là rất nhiều, tuy nhiên các công ty đều đang gặp khó khăn trong vấn đề tiếp cận với kiến trúc hệ thống dữ liệu lớn có thể xử lý thời gian thực, và nền tảng công nghệ vẫn còn đang trong giai đoạn phát triển.

1.8 Giải pháp dữ liệu lớn MapReduce của Google

Năm 2004, Google công bố mô hình xử lý dữ liệu phân tán MapReduce, Mô hình này là sáng kiến của một nhóm các kỹ sư Google, khi nghiên cứu tìm kiếm giải pháp mở rộng cỗ máy tìm kiếm của họ. Có thể coi MapReduce là một mô hình lập trình, hay một giải thuật lập trình, chuyên dùng để giải quyết vấn đề về xử lý dữ liệu lớn. Mô hình này cơ bản gồm hai thao tác chính là Map và Reduce, với ý tưởng là chia công việc lớn ra thành nhiều công việc nhỏ, giao cho nhiều máy tính cùng thực hiện - thao tác Map, sau đó tổng hợp kết quả lại - thao tác Reduce. Trong mô hình này, ngoài hai quá trình cơ bản là **Map** và **Reduce**, còn có thêm hai quá trình nữa là **Split** (phân chia) và **Shuffle** (gom nhóm). Hai

quá trình này lần lượt giữ vai trò phân chia dữ liệu đầu vào, tạo tiền đề cho quá trình Map và gom nhóm dữ liệu đầu ra của quá trình Map để tạo tiền đề cho quá trình Reduce.

MapReduce định nghĩa dữ liệu dưới dạng các cặp <key, value> - <khóa, giá trị>; ví dụ, key có thể là tên của tập tin và value nội dung của tập tin, hoặc key là địa chỉ URL và value là nội dung tại URL, v.v. Dữ liệu được định nghĩa theo dạng này linh hoạt hơn các bảng dữ liệu quan hệ hai chiều truyền thống (quan hệ cha - con hay còn gọi là khóa chính - khóa phụ).

1.8.1 Quá trình Split

Để có thể phân tán công việc trên hệ thống máy tính, trước tiên cần phải phân nhỏ khối dữ liệu đầu vào cần xử lý ra thành nhiều phần, rồi sau đó mới có thể phân công cho mỗi máy xử lý một phần trong số đó. Quá trình phân chia dữ liệu này được gọi là Split, Split sẽ dựa vào một bộ tiêu chí được đặt ra trước để chia nhỏ dữ liệu, mỗi mảnh dữ liệu được chia nhỏ như vậy gọi là một input split.

1.8.2 Quá trình Map và Shuffle

Sau khi các input split được tạo ra, Quá trình Map được thực hiện - hệ thống sẽ phân bổ các input split về các máy xử lý, các máy được phân công sẽ tiếp nhận và xử lý input split được giao, ta gọi quá trình diễn ra trên nội bộ mỗi máy trong quá trình Map là Mapper. Trước khi được xử lý, input split được định dạng lại thành dữ liệu chuẩn của MapReduce - dữ liệu có dạng các cặp <key, value>. Kết thúc quá trình Mapper trên mỗi máy, dữ liệu đầu ra cũng có dạng các cặp <key, value>, chúng sẽ được chuyển sang cho quá trình Shuffle để phân nhóm theo tiêu chí đã được định trước, chuẩn bị cho bước xử lý phân tán tiếp theo. Như vậy, quá trình Shuffle sẽ được thực hiện một cách nội bộ trên mỗi máy chạy Mapper.

1.8.3 Quá trình Reduce

Quá trình Shuffle diễn ra trên nhiều máy nhưng do sử dụng chung một tiêu chí đã được định trước, nên việc phân nhóm dữ liệu trên các máy có sự thống nhất. Các nhóm dữ liệu tương ứng với nhau trên tất cả các máy chạy Shuffle sẽ được gom lại chuyển về cho cùng một máy xử lý, cho ra kết quả cuối cùng. Toàn bộ quá trình này được gọi là Reduce, quá trình xử lý trên từng máy trong quá trình Reduce là quá trình Reducer.

1.8.4 Một số thí dụ ứng dụng mô hình MapReduce

Tìm kiếm theo mẫu (Grep) phân tán

Grep là một tiện ích dòng lệnh dùng cho việc tìm kiếm trên tập dữ liệu văn bản. Khi áp dụng mô hình MapReduce, trong quá trình Map, mỗi Mapper sẽ làm việc với một tập con của tập dữ liệu văn bản, công việc của mỗi Mapper là tìm kiếm và đánh dấu những dòng khớp với biểu thức tìm kiếm trong tập dữ liệu văn bản mà mình phụ trách. Kết quả của các Mapper sẽ được quá trình Reduce gom lại tạo thành kết quả cuối cùng.

Sắp xếp (Sort) phân tán

Mô hình MapReduce rất phù hợp với bài toán sắp xếp dữ liệu. Trong quá trình Map, mỗi Mapper sẽ chỉ giữ nhiệm vụ đọc dữ liệu lên, Shuffle sẽ phân nhóm dữ liệu theo từng khoảng giá trị, Quá trình Reduce sẽ chịu trách nhiệm sắp xếp dữ liệu, mỗi Reducer sẽ sắp xếp dữ liệu trên khoảng giá trị được phân công.

Một ví dụ cụ thể về việc sắp xếp dữ liệu: Bài toán sắp xếp một tập dữ liệu nhiệt độ. Thay vì sắp xếp toàn bộ dữ liệu bằng một chương trình tuần tự, ta có thể xử lý bằng mô hình MapReduce như sau: Tại quá trình Map, dữ liệu sẽ được đọc lên và định dạng thành các cặp <ngày đo, nhiệt độ>. Tại quá trình Shuffle, ta sẽ phân chia dữ liệu theo từng khoảng giá trị của trường “nhiệt độ”, trước khi chuyển qua cho Reduce sắp xếp. Như vậy, mỗi Reducer sẽ chỉ sắp xếp những dữ liệu nằm trong một khoảng nhất định. Ta dễ dàng tổng hợp kết quả sắp xếp của các Reducer, để tạo ra kết quả sắp xếp toàn cục.

1.9 Giải pháp dữ liệu lớn Data Lake của Amazon

Data Lake (Hồ dữ liệu) là thuật ngữ mới của dịch vụ điện toán đám mây Amazon, là mô hình lưu trữ dữ liệu lớn tập trung cho tất cả các loại *dữ liệu* có cấu trúc hay phi cấu trúc, ở mọi quy mô.

Trước tiên, người dùng có thể lưu trữ các dữ liệu mà *không cần* phải xem xét tới cấu trúc dữ liệu, có thể vận hành ngay các mô hình phân tích dữ liệu khác nhau, từ hiển thị bảng biểu, trực quan hóa dữ liệu cho đến xử lý dữ liệu lớn, phân tích dữ liệu trong thời gian thực, ứng dụng học máy để đưa ra các quyết định.

Data Lakes so với Data Warehouses là hai cách tiếp cận khác nhau về lưu trữ và xử lý dữ liệu. Tùy thuộc vào yêu cầu, một tổ chức điển hình sẽ cần cả kho dữ liệu truyền thống và hồ dữ liệu (lớn) vì chúng phục vụ các nhu cầu khác nhau, các trường hợp sử dụng dữ liệu khác nhau. Nếu như **kho dữ liệu** thường là các cơ sở dữ liệu được tối ưu hóa để phân tích dữ liệu quan hệ đến từ các hệ thống giao dịch, các dòng ứng dụng kinh doanh, có cấu trúc dữ liệu và sơ đồ được *xác định*

trước để tối ưu hóa cho các truy vấn SQL nhanh, với kết quả thường được sử dụng để lập báo cáo và phân tích hoạt động, dữ liệu được làm sạch, làm giàu và chuyển đổi thì **hồ dữ liệu** lại đóng vai trò khác.

Hồ dữ liệu thường lưu trữ các dữ liệu đến từ các ứng dụng kinh doanh, dữ liệu không có quan hệ (ứng dụng di động, thiết bị IoT, các phương tiện truyền thông xã hội). Cấu trúc của dữ liệu hoặc sơ đồ dữ liệu thường không được xác định khi dữ liệu được lấy và lưu trữ trong hồ dữ liệu. Điều này có nghĩa là chúng ta có thể lưu trữ **tất cả dữ liệu** mà không cần thiết kế cấu trúc cẩn thận, hoặc cần phải biết trước các câu hỏi hay câu trả lời cần có liên quan tới dữ liệu trong tương lai. Các mô hình phân tích dữ liệu khác nhau như truy vấn SQL, phân tích dữ liệu lớn, tìm kiếm toàn văn bản, phân tích dữ liệu thời gian thực, học máy đều có thể sử dụng **hồ dữ liệu**.

Chương 2. Cơ sở dữ liệu MongoDB

2.1 Cơ sở dữ liệu NoSQL

MongoDB là một cơ sở dữ liệu NoSQL hướng tài liệu (document) được sử dụng để lưu trữ dữ liệu dung lượng lớn. Thay vì sử dụng các bảng và hàng như trong cơ sở dữ liệu quan hệ truyền thống, MongoDB sử dụng các bộ sưu tập (collection) và tài liệu (document). Tài liệu bao gồm các cặp khóa-giá trị là đơn vị dữ liệu cơ bản trong MongoDB. Bộ sưu tập chứa các bộ tài liệu và chức năng tương đương với các bảng cơ sở dữ liệu quan hệ. MongoDB xuất hiện vào khoảng giữa những năm 2000 và đã trở thành mô hình lưu trữ dữ liệu lớn khá phổ biến.

2.2 Các tính năng của MongoDB

Mỗi cơ sở dữ liệu chứa các bộ sưu tập lần lượt chứa các tài liệu. Mỗi tài liệu có thể khác nhau với một số trường khác nhau. Kích thước và nội dung của mỗi tài liệu có thể khác nhau.

Cấu trúc tài liệu phù hợp hơn với cách các nhà phát triển xây dựng các lớp và đối tượng của họ bằng ngôn ngữ lập trình tương ứng của họ. Các nhà phát triển thường sẽ nói rằng các lớp của họ không phải là hàng và cột mà có cấu trúc rõ ràng với các cặp khóa-giá trị.

Các hàng (hoặc tài liệu như được gọi trong MongoDB) không cần phải có một lược đồ được xác định trước. Thay vào đó, các trường có thể được tạo nhanh chóng.

Mô hình dữ liệu có sẵn trong MongoDB cho phép bạn biểu diễn các mối quan hệ phân cấp, lưu trữ mảng và các cấu trúc phức tạp khác dễ dàng hơn.

Khả năng mở rộng - Môi trường MongoDB rất có thể mở rộng. Các công ty trên toàn thế giới đã xác định các cụm với một số trong số họ chạy hơn 100 nút với khoảng hàng triệu tài liệu trong cơ sở dữ liệu

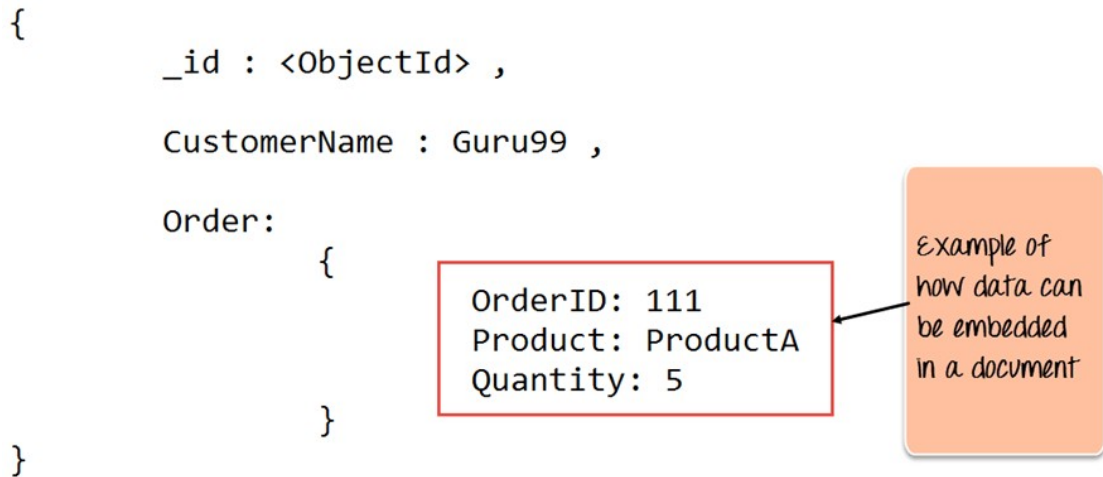
2.3 Ví dụ về MongoDB

Ví dụ dưới đây cho thấy cách một tài liệu có thể được mô hình hóa trong MongoDB như thế nào.

Trường `_id` được MongoDB thêm vào để xác định duy nhất tài liệu trong bộ sưu tập.

Những gì bạn có thể lưu ý là Dữ liệu đặt hàng (OrderID, Sản phẩm và Số

lượng) trong RDBMS thường sẽ được lưu trữ trong một bảng riêng biệt, trong khi trong MongoDB, nó thực sự được lưu trữ dưới dạng tài liệu nhúng trong chính bộ sưu tập. Đây là một trong những điểm khác biệt chính về cách dữ liệu được mô hình hóa trong MongoDB.



2.4 Các thành phần chính của kiến trúc MongoDB

Dưới đây là một số thuật ngữ phổ biến được sử dụng trong MongoDB

_id - Đây là trường bắt buộc trong mọi tài liệu MongoDB. Trường **_id** đại diện cho một giá trị duy nhất trong tài liệu MongoDB. Trường **_id** giống như khóa chính của tài liệu. Nếu bạn tạo một tài liệu mới mà không có trường **_id**, MongoDB sẽ tự động tạo trường. Vì vậy, ví dụ: nếu chúng ta thấy ví dụ về bảng khách hàng ở trên, Mongo DB sẽ thêm số nhận dạng duy nhất gồm 24 chữ số vào mỗi tài liệu trong bộ sưu tập.

_Id CustomerID CustomerName OrderID

563479cc8a8a4246bd27d784 11 Guru99 111

563479cc7a8a4246bd47d784 22 Trevor Smith 222

563479cc9a8a4246bd57d784 33 Nicole 333

Bộ sưu tập (Collection) - Đây là một nhóm các tài liệu MongoDB. Tập hợp tương đương với một bảng được tạo trong bất kỳ RDMS nào khác như Oracle hoặc MS SQL. Một bộ sưu tập tồn tại trong một cơ sở dữ liệu duy nhất. Như đã thấy từ các bộ sưu tập giới thiệu không thực thi bất kỳ loại cấu trúc nào.

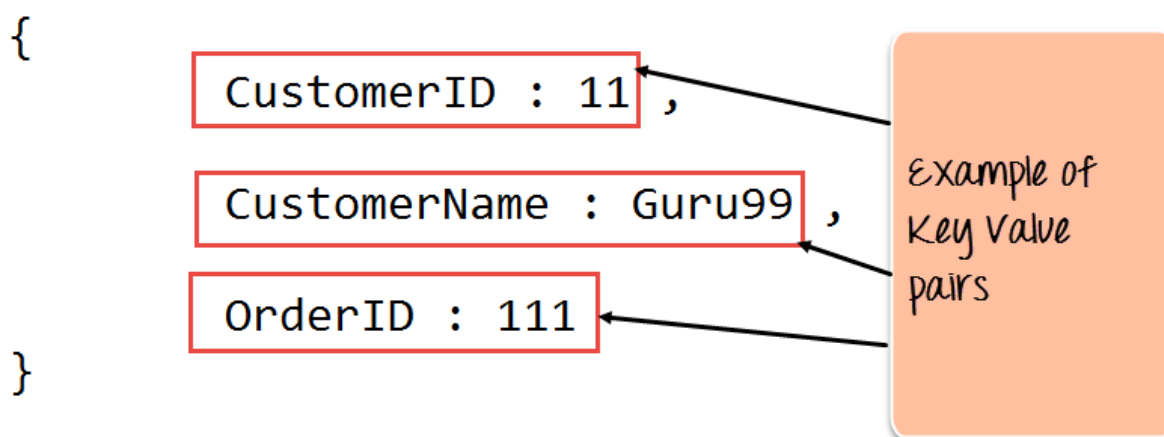
Con trỏ (Cursor) - Đây là một con trỏ đến tập kết quả của một truy vấn. Khách hàng có thể lặp lại qua con trỏ để truy xuất kết quả.

Cơ sở dữ liệu (Database) - Đây là vùng chứa cho các bộ sưu tập như trong RDMS, trong đó nó là vùng chứa cho các bảng. Mỗi cơ sở dữ liệu có một bộ tệp riêng trên hệ thống tệp. Một máy chủ MongoDB có thể lưu trữ nhiều cơ sở dữ liệu.

Tài liệu (Document) - Bản ghi trong bộ sưu tập MongoDB về cơ bản được gọi là tài liệu. Đến lượt mình, tài liệu sẽ bao gồm tên trường và các giá trị.

Trường (Field) - Một cặp tên-giá trị trong tài liệu. Một tài liệu có không hoặc nhiều trường. Các trường tương tự như các cột trong cơ sở dữ liệu quan hệ.

Sơ đồ sau đây cho thấy một ví dụ về Trường với các cặp giá trị Khóa. Vì vậy, trong ví dụ bên dưới, CustomerID và 11 là một trong những cặp giá trị khóa được xác định trong tài liệu.



JSON - Đây được gọi là Ký hiệu đối tượng JavaScript. Đây là định dạng văn bản thuần túy, có thể đọc được của con người để thể hiện dữ liệu có cấu trúc. JSON hiện được hỗ trợ trong nhiều ngôn ngữ lập trình.

Chỉ cần một ghi chú nhanh về sự khác biệt chính giữa trường `_id` và trường thập thông thường. Trường `_id` được sử dụng để xác định duy nhất các tài liệu trong bộ sưu tập và được MongoDB tự động thêm vào khi bộ sưu tập được tạo.

2.5 Tại sao sử dụng MongoDB?

Dưới đây là một số lý do tại sao nên bắt đầu sử dụng MongoDB

Hướng tài liệu (document-oriented) - Vì MongoDB là cơ sở dữ liệu kiểu NoSQL, thay vì có dữ liệu ở định dạng kiểu quan hệ, nó sẽ lưu trữ dữ liệu trong tài liệu. Điều này làm cho MongoDB rất linh hoạt và thích ứng với tình hình và yêu cầu kinh doanh thực tế.

Truy vấn đặc biệt (Ad hoc queries) - MongoDB hỗ trợ tìm kiếm theo trường, truy vấn phạm vi và tìm kiếm biểu thức chính quy. Các truy vấn có thể được thực hiện để trả về các trường cụ thể trong tài liệu.

Lập chỉ mục (Indexing) - Các chỉ mục có thể được tạo để cải thiện hiệu suất của các tìm kiếm trong MongoDB. Bất kỳ trường nào trong tài liệu MongoDB đều có thể được lập chỉ mục.

Nhân bản (Replication) - MongoDB có thể cung cấp tính khả dụng cao với các tập hợp bản sao. Một tập hợp bản sao bao gồm hai hoặc nhiều cá thể DB mongo. Mỗi thành viên của nhóm bản sao có thể hoạt động trong vai trò của bản sao chính hoặc thứ cấp bất kỳ lúc nào. Bản sao chính là máy chủ chính tương tác với máy khách và thực hiện tất cả các hoạt động đọc / ghi. Các bản sao thứ cấp duy trì một bản sao dữ liệu của bản sao chính bằng cách sử dụng bản sao cài sẵn. Khi một bản sao chính bị lỗi, tập hợp bản sao sẽ tự động chuyển sang bản sao thứ cấp và sau đó nó trở thành máy chủ chính.

Cân bằng tải (Load balancing) - MongoDB sử dụng khái niệm sharding để chia tỷ lệ theo chiều ngang bằng cách chia nhỏ dữ liệu trên nhiều phiên bản MongoDB. MongoDB có thể chạy trên nhiều máy chủ, cân bằng tải và / hoặc sao chép dữ liệu để giữ cho hệ thống hoạt động trong trường hợp lỗi phần cứng.

2.6 Mô hình hóa dữ liệu trong MongoDB

Như chúng ta đã thấy, dữ liệu trong MongoDB có một lược đồ linh hoạt. Không giống như trong cơ sở dữ liệu SQL, nơi phải khai báo lược đồ của bảng trước khi chèn dữ liệu, các bộ sưu tập của MongoDB không thực thi cấu trúc tài liệu. Loại linh hoạt này là điều khiến MongoDB trở nên mạnh mẽ.

Khi lập mô hình dữ liệu trong Mongo, hãy ghi nhớ những điều sau

Các nhu cầu của ứng dụng là gì - Nhìn vào nhu cầu kinh doanh của ứng dụng và xem dữ liệu nào và loại dữ liệu cần thiết cho ứng dụng. Dựa trên điều này, đảm bảo rằng cấu trúc của tài liệu được quyết định phù hợp.

Các mẫu truy xuất dữ liệu là gì - Nếu bạn thấy trước việc sử dụng truy vấn nhiều thì hãy xem xét việc sử dụng các chỉ mục trong mô hình dữ liệu của mình để cải thiện hiệu quả của các truy vấn.

Có thường xuyên chèn, cập nhật và xóa trong cơ sở dữ liệu không? Xem xét lại việc sử dụng các chỉ mục hoặc kết hợp sharding nếu được yêu cầu trong thiết kế mô hình dữ liệu của bạn để cải thiện hiệu quả của môi trường MongoDB

tổng thể của bạn.

2.7 Sự khác biệt giữa MongoDB và RDBMS

Dưới đây là một số khác biệt về thuật ngữ chính giữa MongoDB và RDBMS

RDBMS	MongoDB	Sự khác biệt
Table	Collection	Trong RDBMS, bảng chứa các cột và hàng được sử dụng để lưu trữ dữ liệu trong khi trong MongoDB, cấu trúc tương tự này được gọi là tập hợp. Tập hợp chứa các tài liệu lần lượt chứa các Trường, lần lượt là các cặp khóa-giá trị.
Row	Document	Trong RDBMS, hàng đại diện cho một mục dữ liệu có cấu trúc ngầm định trong một bảng. Trong MongoDB, dữ liệu được lưu trữ trong các tài liệu.
Column	Field	Trong RDBMS, cột biểu thị một tập hợp các giá trị dữ liệu. Chúng trong MongoDB được gọi là Trường.
Joins	Embedded documents	Trong RDBMS, dữ liệu đôi khi được trải rộng trên nhiều bảng khác nhau và để hiển thị toàn bộ dữ liệu, một phép nối đôi khi được hình thành giữa các bảng để lấy dữ liệu. Trong MongoDB, dữ liệu thường được lưu trữ trong một tập hợp duy nhất, nhưng được phân tách bằng cách sử dụng tài liệu nhúng. Vì vậy, không có khái niệm tham gia trong MongoDB.

Ngoài sự khác biệt về điều khoản, một số khác biệt khác được hiển thị bên dưới

1. Cơ sở dữ liệu quan hệ được biết đến với việc thực thi tính toàn vẹn của dữ liệu. Đây không phải là một yêu cầu rõ ràng trong MongoDB.

2. RDBMS yêu cầu dữ liệu phải được chuẩn hóa trước để nó có thể ngăn chặn các bản ghi mờ côi và bản sao Việc chuẩn hóa dữ liệu sau đó yêu cầu nhiều bảng hơn, sau đó sẽ dẫn đến nhiều phép nối bảng hơn, do đó yêu cầu nhiều khóa và chỉ mục hơn.

Khi cơ sở dữ liệu bắt đầu phát triển, hiệu suất có thể bắt đầu trở thành một vấn đề. Một lần nữa, đây không phải là một yêu cầu rõ ràng trong MongoDB. MongoDB linh hoạt và không cần dữ liệu được chuẩn hóa trước.

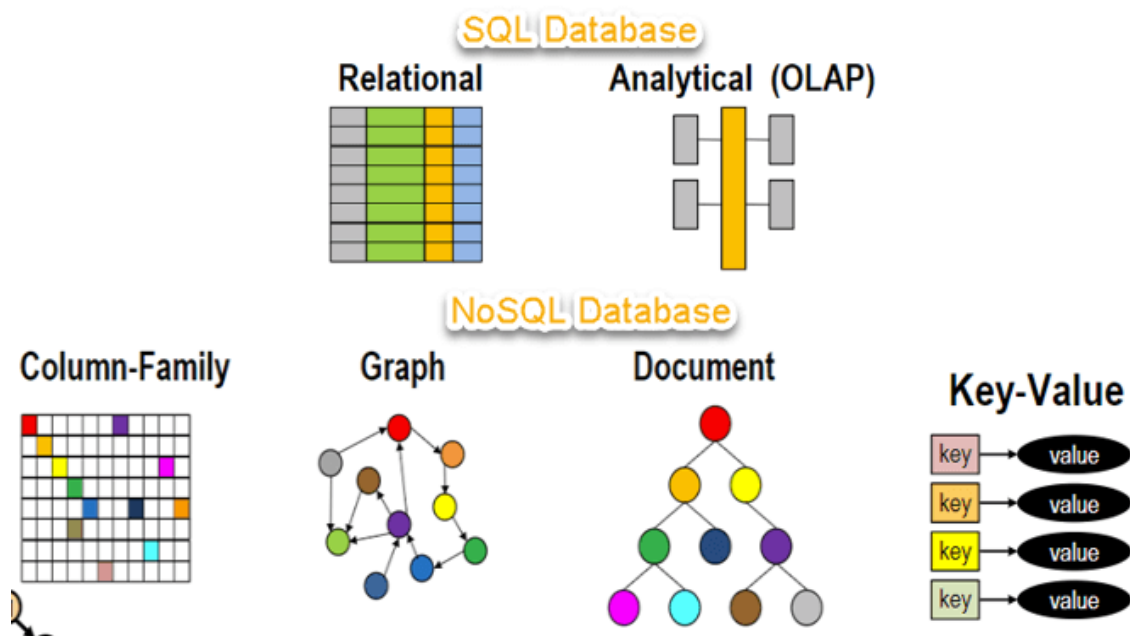
2.8 Hướng dẫn các loại cơ sở dữ liệu NoSQL

2.8.1 NoSQL là gì?

Cơ sở dữ liệu NoSQL là một Hệ thống quản lý dữ liệu không quan hệ, không yêu cầu một lược đồ cố định. Nó tránh tham gia và dễ mở rộng. Mục đích chính của việc sử dụng cơ sở dữ liệu NoSQL là dành cho các kho dữ liệu phân tán với nhu cầu lưu trữ dữ liệu lớn. NoSQL được sử dụng cho Dữ liệu lớn và ứng dụng web thời gian thực. Ví dụ, các công ty như Twitter, Facebook và Google thu thập hàng terabyte dữ liệu người dùng mỗi ngày.

Cơ sở dữ liệu NoSQL là viết tắt của "Not Only SQL" hoặc "Not SQL". Mặc dù một thuật ngữ tốt hơn sẽ là "NoREL", NoSQL đã bắt kịp. Carl Strozzi đã giới thiệu khái niệm NoSQL vào năm 1998.

RDBMS truyền thống sử dụng cú pháp SQL để lưu trữ và truy xuất dữ liệu để có thêm thông tin chi tiết. Thay vào đó, hệ thống cơ sở dữ liệu NoSQL bao gồm một loạt các công nghệ cơ sở dữ liệu có thể lưu trữ dữ liệu có cấu trúc, bán cấu trúc, phi cấu trúc và đa hình. Hãy hiểu về NoSQL với một sơ đồ trong hướng dẫn cơ sở dữ liệu NoSQL này:



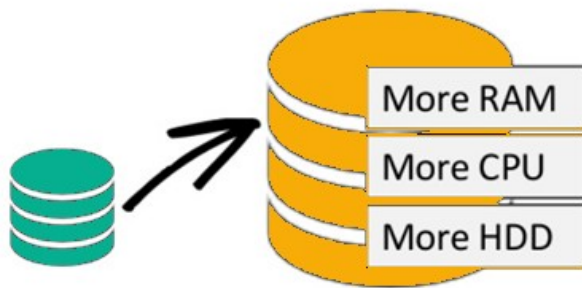
2.8.2 Tại sao NoSQL?

Khái niệm cơ sở dữ liệu NoSQL trở nên phổ biến với những gã khổng lồ Internet như Google, Facebook, Amazon, v.v., những người xử lý khối lượng dữ liệu khổng lồ. Thời gian phản hồi của hệ thống trở nên chậm khi bạn sử dụng RDBMS cho khối lượng lớn dữ liệu.

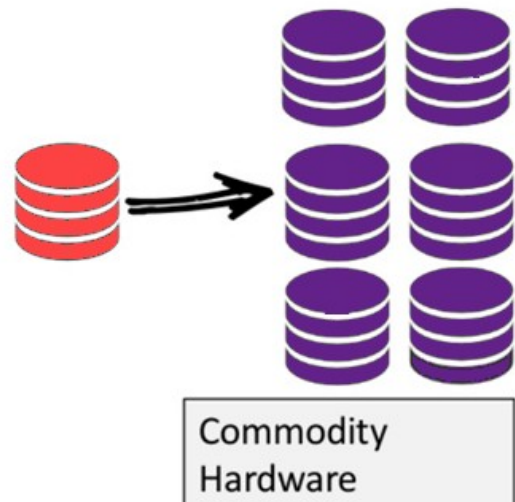
Để giải quyết vấn đề này, ta có thể "mở rộng theo quy mô (Scale-Up)" hệ thống của mình bằng cách nâng cấp phần cứng hiện có của chúng tôi. Quá trình này là tốn kém.

Giải pháp thay thế cho vấn đề này là tái phân phối tải các cơ sở dữ liệu trên nhiều máy chủ bất cứ khi nào tải tăng lên. Phương pháp này được gọi là "mở rộng theo phạm vi (Scale-Out)".

Scale-Up (vertical scaling):



Scale-Out (horizontal scaling):



Cơ sở dữ liệu NoSQL không quan hệ, vì vậy nó mở rộng theo phạm vi tốt hơn cơ sở dữ liệu quan hệ vì chúng được thiết kế với các ứng dụng web.

2.8.3 Các tính năng của NoSQL

Không quan hệ

Cơ sở dữ liệu NoSQL không bao giờ tuân theo mô hình quan hệ

Không bao giờ cung cấp bảng có bản ghi cột cố định phẳng

Làm việc với các tập hợp hoặc BLOB độc lập

Không yêu cầu ánh xạ quan hệ đối tượng và chuẩn hóa dữ liệu

Không có các tính năng phức tạp như ngôn ngữ truy vấn, trình lập kế hoạch truy vấn, tham chiếu toàn vẹn tham chiếu, ACID

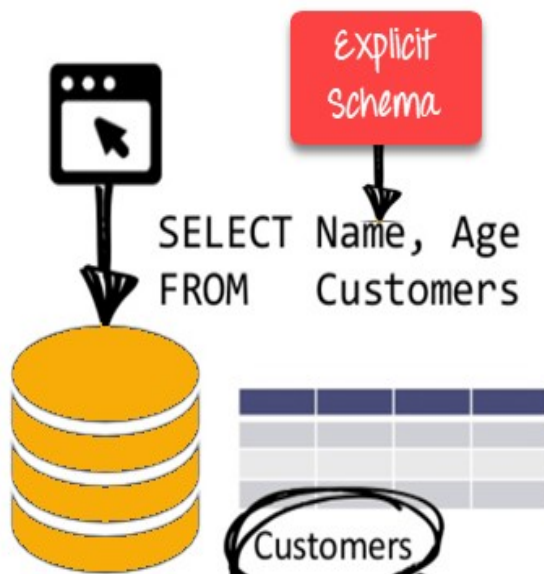
Không có giản đồ

Cơ sở dữ liệu NoSQL hoặc không có lược đồ hoặc có lược đồ thoải mái

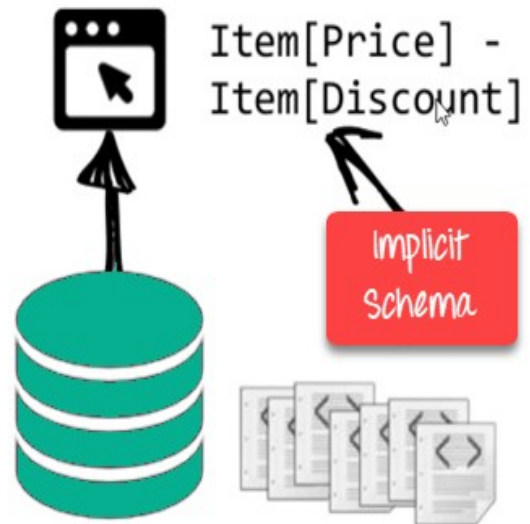
Không yêu cầu bất kỳ loại định nghĩa nào về lược đồ dữ liệu

Cung cấp cấu trúc dữ liệu không đồng nhất trong cùng một miền

RDBMS:



NoSQL DB:

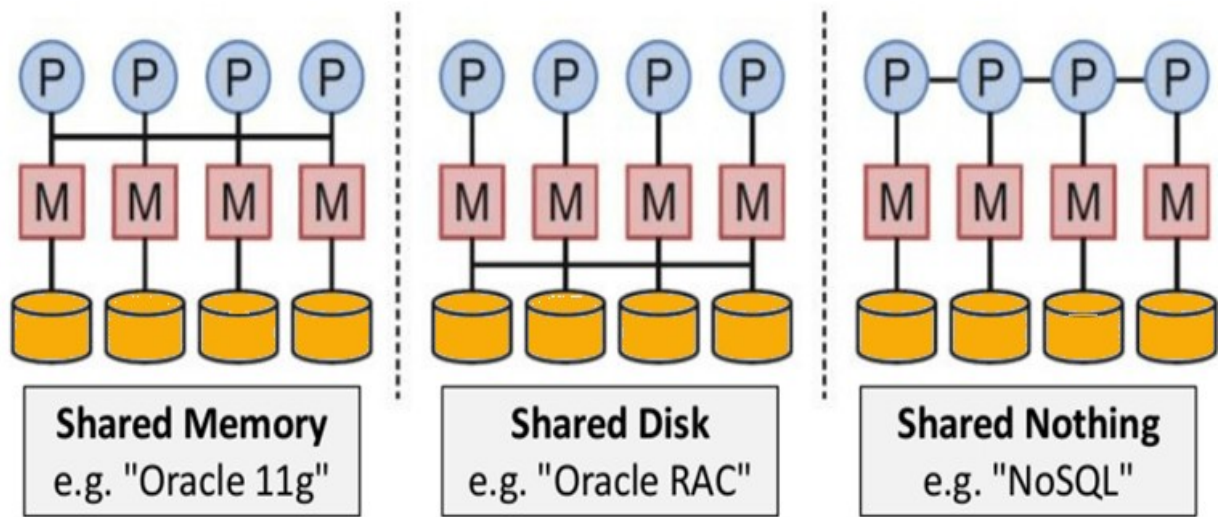


2.8.4 API đơn giản

- + Cung cấp giao diện dễ sử dụng để lưu trữ và truy vấn dữ liệu được cung cấp API cho phép các phương pháp lựa chọn và thao tác dữ liệu cấp thấp.
- + Các giao thức dựa trên văn bản chủ yếu được sử dụng với HTTP REST với JSON
- + Hầu như không được sử dụng ngôn ngữ truy vấn NoSQL dựa trên tiêu chuẩn
- + Cơ sở dữ liệu hỗ trợ web chạy dưới dạng dịch vụ trực tuyến

2.8.5 Phân phối

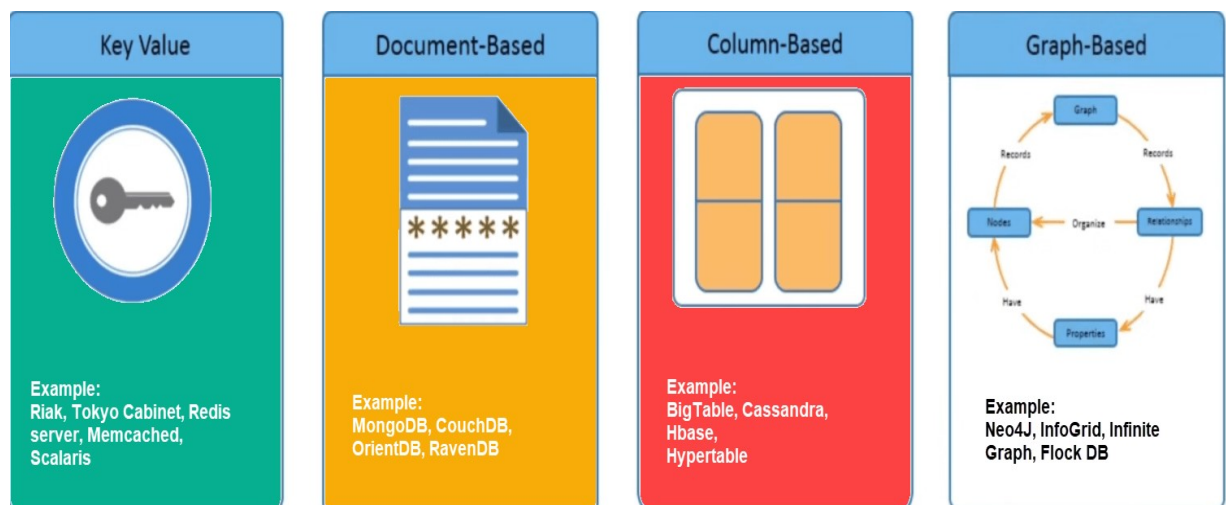
- + Nhiều cơ sở dữ liệu NoSQL có thể được thực thi theo kiểu phân tán
- + Cung cấp khả năng tự động mở rộng quy mô và khả năng vượt qua lỗi
- + Thường thì khái niệm ACID có thể được hy sinh cho khả năng mở rộng và thông lượng
- + Chủ yếu là không có sự sao chép đồng bộ giữa các nút được phân phối Nhân bản đa chủ không đồng bộ, ngang hàng, Nhân bản HDFS
- + Chỉ cung cấp tính nhất quán cuối cùng
- + Kiến trúc Không chia sẻ gì. Điều này cho phép phối hợp ít hơn và phân phối cao hơn.



2.8.6 Các loại cơ sở dữ liệu NoSQL

Cơ sở dữ liệu NoSQL chủ yếu được phân loại thành bốn loại: Cặp khóa-giá trị, Định hướng cột, Dựa trên đồ thị và Hướng tài liệu. Mỗi danh mục đều có các thuộc tính và giới hạn riêng của nó. Không có cơ sở dữ liệu nào được chỉ định ở trên là tốt hơn để giải quyết tất cả các vấn đề. Người dùng nên chọn cơ sở dữ liệu dựa trên nhu cầu sản phẩm của họ.

- + Dựa trên cặp khóa-giá trị
- + Biểu đồ hướng cột
- + Dựa trên đồ thị
- + Định hướng tài liệu



Dựa trên cặp giá trị chính

Dữ liệu được lưu trữ trong các cặp khóa / giá trị. Nó được thiết kế theo cách để

xử lý nhiều dữ liệu và tải nặng.

Cơ sở dữ liệu lưu trữ cặp khóa-giá trị lưu trữ dữ liệu dưới dạng bảng băm trong đó mỗi khóa là duy nhất và giá trị có thể là JSON, BLOB (Đối tượng lớn nhị phân), chuỗi, v.v.

Ví dụ: một cặp khóa-giá trị có thể chứa một khóa như "Trang web" được liên kết với một giá trị như "Guru99".

Key	Value
Name	Joe Bloggs
Age	42
Occupation	Stunt Double
Height	175cm
Weight	77kg

Đây là một trong những ví dụ cơ sở dữ liệu NoSQL cơ bản nhất. Loại cơ sở dữ liệu NoSQL này được sử dụng như một bộ sưu tập, từ điển, mảng kết hợp, v.v. Kho giá trị chính giúp nhà phát triển lưu trữ dữ liệu không có lược đồ. Chúng hoạt động tốt nhất cho nội dung giỏ hàng.

Redis, Dynamo, Riak là một số ví dụ NoSQL về DataBases lưu trữ giá trị-khóa. Tất cả chúng đều dựa trên giấy Dynamo của Amazon.

Dựa trên cột

Cơ sở dữ liệu hướng theo cột hoạt động trên các cột và dựa trên tài liệu BigTable của Google. Mỗi cột được xử lý riêng biệt. Giá trị của cơ sở dữ liệu cột đơn được lưu trữ liên kế.

ColumnFamily			
Row Key	Column Name		
	Key	Key	Key
	Value	Value	Value
	Column Name		
	Key	Key	Key
	Value	Value	Value

Cơ sở dữ liệu NoSQL dựa trên cột

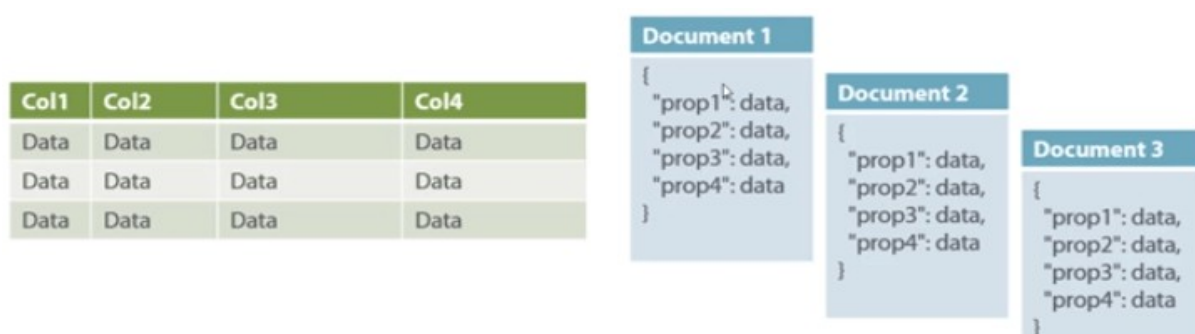
Chúng mang lại hiệu suất cao trên các truy vấn tổng hợp như SUM, COUNT, AVG, MIN, v.v. vì dữ liệu có sẵn trong một cột.

Cơ sở dữ liệu NoSQL dựa trên cột được sử dụng rộng rãi để quản lý kho dữ liệu, kinh doanh thông minh, CRM, Danh mục thẻ thư viện,

HBase, Cassandra, HBase, Hypertable là các ví dụ truy vấn NoSQL của cơ sở dữ liệu dựa trên cột.

Định hướng tài liệu

Cơ sở dữ liệu NoSQL hướng tài liệu lưu trữ và truy xuất dữ liệu dưới dạng một cặp giá trị khóa nhưng phần giá trị được lưu trữ dưới dạng tài liệu. Tài liệu được lưu trữ ở định dạng JSON hoặc XML. Giá trị được hiểu bởi DB và có thể được truy vấn.



Trong sơ đồ bên trái này, bạn có thể thấy chúng ta có các hàng và cột, và ở bên phải, chúng ta có một cơ sở dữ liệu tài liệu có cấu trúc tương tự như JSON. Bây giờ đối với cơ sở dữ liệu quan hệ, bạn phải biết bạn có những cột nào, v.v. Tuy nhiên, đối với cơ sở dữ liệu tài liệu, bạn có kho dữ liệu như đối tượng JSON. Bạn không cần phải xác định cái nào làm cho nó linh hoạt.

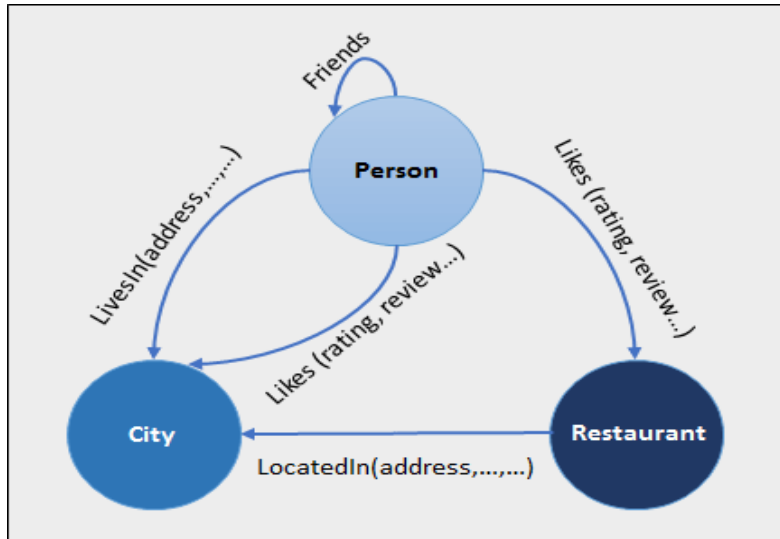
Loại tài liệu này chủ yếu được sử dụng cho các hệ thống CMS, nền tảng blog, phân tích thời gian thực và các ứng dụng thương mại điện tử. Nó không nên sử dụng cho các giao dịch phức tạp yêu cầu nhiều hoạt động hoặc truy vấn dựa trên các cấu trúc tổng hợp khác nhau.

Amazon SimpleDB, CouchDB, MongoDB, Riak, Lotus Notes, MongoDB, là các hệ thống DBMS có nguồn gốc tài liệu phổ biến.

Dựa trên đồ thị

Cơ sở dữ liệu kiểu đồ thị lưu trữ các thực thể cũng như các mối quan hệ giữa các thực thể đó. Thực thể được lưu trữ dưới dạng một nút với mỗi quan hệ là các cạnh. Một cạnh cho biết mối quan hệ giữa các nút. Mỗi nút và cạnh có một

mã định danh duy nhất.



So với cơ sở dữ liệu quan hệ nơi các bảng được kết nối với nhau một cách lỏng lẻo, cơ sở dữ liệu Đồ thị có bản chất là đa quan hệ. Mỗi quan hệ truyền tải nhanh chóng vì chúng đã được ghi lại vào DB và không cần phải tính toán chúng.

Cơ sở dữ liệu đồ thị chủ yếu được sử dụng cho mạng xã hội, hậu cần, dữ liệu không gian.

2.8.7 Công cụ cơ chế truy vấn cho NoSQL

Cơ chế truy xuất dữ liệu phổ biến nhất là truy xuất dựa trên REST của một giá trị dựa trên khóa / ID của nó với tài nguyên GET

Kho tài liệu Cơ sở dữ liệu cung cấp các truy vấn khó hơn khi chúng hiểu giá trị trong cặp khóa-giá trị. Ví dụ, CouchDB cho phép xác định các chế độ xem với MapReduce

2.9 Ưu điểm của NoSQL

- + Có thể được sử dụng làm nguồn dữ liệu chính hoặc phân tích
- + Khả năng dữ liệu lớn
- + Không có điểm thất bại nào
- + Nhân rộng dễ dàng
- + Không cần lớp đệm riêng biệt
- + Cung cấp hiệu suất nhanh chóng và khả năng mở rộng theo chiều ngang.

- + Có thể xử lý dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc với hiệu quả như nhau
- + Lập trình hướng đối tượng dễ sử dụng và linh hoạt
- + Cơ sở dữ liệu NoSQL không cần máy chủ hiệu suất cao chuyên dụng
- + Hỗ trợ nền tảng và ngôn ngữ chính dành cho nhà phát triển
- + Thực hiện đơn giản hơn so với sử dụng RDBMS
- + Có thể đóng vai trò là nguồn dữ liệu chính cho các ứng dụng trực tuyến.
- + Xử lý dữ liệu lớn quản lý tốc độ, sự đa dạng, khối lượng và độ phức tạp của dữ liệu
- + Vượt trội ở cơ sở dữ liệu phân tán và hoạt động của trung tâm đa dữ liệu
- + Loại bỏ nhu cầu về một lớp bộ nhớ đệm cụ thể để lưu trữ dữ liệu
- + Cung cấp thiết kế giản đồ linh hoạt có thể dễ dàng thay đổi mà không có thời gian chết hoặc gián đoạn dịch vụ

2.10 Nhược điểm của NoSQL

- + Không có quy tắc tiêu chuẩn hóa
- + Khả năng truy vấn hạn chế
- + Cơ sở dữ liệu và công cụ RDBMS tương đối hoàn thiện
- + Nó không cung cấp bất kỳ khả năng cơ sở dữ liệu truyền thống nào, như tính nhất quán khi nhiều giao dịch được thực hiện đồng thời.
- + Khi khối lượng dữ liệu tăng lên, rất khó để duy trì các giá trị duy nhất vì các khóa trở nên khó khăn
- + Không hoạt động tốt với dữ liệu quan hệ
- + Đường cong học tập khó đối với các nhà phát triển mới
- + Các tùy chọn mã nguồn mở nên không quá phổ biến đối với các doanh nghiệp.

2.11 Tóm lược

NoSQL là một DMS không quan hệ, không yêu cầu lược đồ cố định, tránh các phép nối và dễ mở rộng

Khái niệm cơ sở dữ liệu NoSQL trở nên phổ biến với những gã khổng lồ Internet như Google, Facebook, Amazon, v.v., những người xử lý khối lượng dữ liệu khổng lồ

Vào năm 1998- Carlo Strozzi sử dụng thuật ngữ NoSQL cho cơ sở dữ liệu quan hệ mã nguồn mở, nhẹ của mình

Cơ sở dữ liệu NoSQL không bao giờ tuân theo mô hình quan hệ, nó không có lược đồ hoặc có lược đồ thoải mái

Bốn loại Cơ sở dữ liệu NoSQL là 1). Dựa trên cặp giá trị khóa 2). Đồ thị hướng cột 3). Đồ thị dựa trên 4).

NOSQL có thể xử lý dữ liệu có cấu trúc, bán cấu trúc và phi cấu trúc với hiệu quả như nhau

NOSQL cung cấp khả năng truy vấn hạn chế

Chương 3 Cụm phân tích dữ liệu lớn Hadoop

3.1 Giới thiệu

Hadoop là giải pháp cho các vấn đề trên Big Data. Đây là công nghệ lưu trữ bộ dữ liệu khổng lồ trên một cụm máy giá rẻ theo cách phân tán. Không chỉ vậy, nó cung cấp phân tích Dữ liệu lớn thông qua khuôn khổ điện toán phân tán. Nó là một phần mềm mã nguồn mở được phát triển như một dự án của Apache Software Foundation.

Hadoop được Doug Cutting tạo ra khi ông nghiên cứu về Nutch - một ứng dụng tìm kiếm. Hadoop được viết bằng Java, dùng hỗ trợ xây dựng, thực thi các ứng dụng tính toán phân tán theo mô hình MapReduce. Hadoop cluster là hệ thống máy tính đã được triển khai nền tảng Hadoop, một Hadoop cluster bao gồm hai thành phần cơ bản là kiến trúc MapReduce và hệ thống tập tin phân tán HDFS, Trong đó:

- Kiến trúc MapReducer gồm hai phần: TaskTracker - trực tiếp thực thi các tác vụ xử lý dữ liệu, JobTracker - quản lý và phân chia công việc cho các TaskTracker.
- Hệ thống HDFS gồm hai phần: DataNode - nơi trực tiếp lưu trữ dữ liệu, mỗi DataNode chịu trách nhiệm lưu trữ một phần dữ liệu của hệ thống, NameNode - quản lý các DataNode, dẫn đường cho các yêu cầu truy xuất dữ liệu.

Kiến trúc của Hadoop cluster là kiến trúc Master-Slave, và cả hai thành phần MapReduce và HDFS đều tuân theo kiến trúc này.

Vào năm 2008, Yahoo đã trao Hadoop cho Apache Software Foundation. Kể từ đó, hai phiên bản Hadoop đã ra đời. Phiên bản 1.0 vào năm 2011 và phiên bản 2.0.6 vào năm 2013. Hadoop có nhiều loại khác nhau như Cloudera, IBM BigInsight, MapR và Hortonworks.

3.2 MapReduce Layer

JobTracker và TaskTracker

Trong Hadoop, mỗi quá trình xử lý MapReduce được gọi là một job. Việc thực hiện job sẽ được quản lý bởi hai đối tượng là JobTracker và TaskTracker. JobTracker hoạt động tại máy master có nhiệm vụ quản lý toàn bộ hệ thống gồm việc tạo và quản lý job, phân bố dữ liệu và phân công công việc cho các TaskTracker, xử lý lỗi, v.v. Tại mỗi máy slave có một TaskTracker hoạt động để

tạo các task xử lý theo yêu cầu của JobTracker. Ngoài ra, định kỳ mỗi khoảng thời gian, TaskTracker phải gửi tín hiệu HeartBeat về JobTracker để thông báo rằng nó vẫn đang còn hoạt động. Điều này đảm bảo JobTracker lập thời biểu công việc chính xác và hiệu quả cho cả hệ thống.

Cơ chế hoạt động mô hình MapReduce trong Hadoop

Mỗi khi có yêu cầu thực thi một ứng dụng MapReduce, JobTracker sẽ tạo ra một JobClient và chép toàn bộ code thực thi cần thiết của job đó lên hệ thống tập tin phân tán HDFS, mỗi JobClient sẽ được gán một jobID duy nhất. Tiếp theo JobClient sẽ gửi một yêu cầu thực thi job lên JobTracker, JobTracker dựa theo yêu cầu của JobClient, sẽ gửi yêu cầu khởi tạo task kèm theo các thông tin phân công công việc đến các TaskTracker. Mỗi TaskTracker sẽ dựa vào thông tin phân công lần lượt thực hiện: Khởi tạo map task hoặc reduce task, chép toàn bộ code thực thi trên HDFS về, thực hiện công việc được phân công. Sau khi thực hiện xong, TaskTracker sẽ thông báo cho JobTracker và tự giải phóng.

3.3 Hadoop Distributed File System (HDFS)

3.3.1 Định nghĩa của HDFS

Hadoop Distributed File System (HDFS) là một hệ thống tập tin phân tán, được thiết kế để chạy trên hệ thống nhiều máy tính được nối mạng với nhau, có khả năng chịu lỗi cao và có thể triển khai trên hệ thống phần cứng không đòi hỏi cấu hình đắt tiền. Có rất nhiều đặc điểm giống nhau giữa HDFS và những hệ thống tập tin phân tán khác. Tuy nhiên, HDFS có những đặc điểm nổi bật riêng giúp nó có khả năng hỗ trợ tốt cho các ứng dụng xử lý dữ liệu lớn.

3.3.2 Đặc điểm của HDFS

Dữ liệu lưu trữ cực lớn

HDFS được thiết kế để lưu trữ những tập tin với kích thước hàng trăm megabyte, gigabyte hay terabyte. Ngày nay, hệ thống HDFS có thể lưu trữ lên đến petabyte dữ liệu.

Xử lý lỗi phần cứng

HDFS không đòi hỏi phần cứng cấu hình cao đối với hệ thống máy tính. Vì vậy, việc xảy ra lỗi trên các thiết bị phần cứng là hoàn toàn có thể xảy ra một cách thường xuyên. Một hệ thống HDFS có thể bao gồm hàng ngàn máy xử lý, mỗi máy (node) lưu trữ một phần của dữ liệu. HDFS có cơ chế quản lý toàn bộ các node đang chạy trên hệ thống để nhận biết node nào đang rảnh và node nào bị

lỗi. Những công việc và dữ liệu được xử lý tại node bị lỗi đó sẽ được chuyển sang node rảnh của hệ thống để xử lý lại.

Dữ liệu chặt chẽ

HDFS hoạt động theo cơ chế ghi một lần - đọc nhiều lần. Mỗi tập tin sẽ được tạo, ghi dữ liệu và đóng lại hoàn toàn. Việc cập nhật ghi thêm dữ liệu vào tập tin là không thể thực hiện trên HDFS. Dữ liệu có thể được truy xuất nhiều lần nhưng vẫn đảm bảo tính nhất quán. Cơ chế thích hợp cho những ứng dụng đọc dữ liệu theo dạng tuần tự.

Di chuyển tính toán thay vì di chuyển dữ liệu

Những yêu cầu tính toán của ứng dụng sẽ được thực hiện tại node chứa dữ liệu gần nhất với nó. Điều này càng hiệu quả đối với dữ liệu lớn và hệ thống mạng băng thông hẹp. HDFS cung cấp giao diện cho ứng dụng tìm kiếm và di chuyển chính nó đến vị trí dữ liệu gần nhất.

Chạy trên nhiều nền tảng và thiết bị

HDFS được thiết kế để dễ dàng di chuyển từ nền tảng này sang nền tảng khác, thiết bị này sang thiết bị khác. Điều này tạo điều kiện thuận lợi cho việc ứng dụng HDFS một cách rộng rãi.

3.3.3 Các khái niệm chính của HDFS

Block

Mỗi đĩa cứng có một kích thước block nhất định. Đó là kích thước dữ liệu nhỏ nhất có thể được ghi và đọc trên đó. Kích thước block cho những tập tin hệ thống cho những đĩa lưu trữ đơn thường khoảng vài kilobyte. Việc này giúp người dùng dễ dàng đọc hoặc ghi tập tin với chiều dài đó. HDFS cũng có quy định về kích thước block. Mặc định mỗi block trên HDFS có kích thước là 64MB.

NameNode và DataNode

HDFS là một kiến trúc Master/Slave. HDFS cluster là hệ thống máy tính đã được triển khai HDFS. Trong một cluster HDFS có duy nhất một NameNode đóng vai trò giống như một master, dùng quản lý không gian tên của hệ thống tập tin (file system namespace) và điều phối việc truy xuất dữ liệu. Ngoài ra, còn có các DataNode đóng vai trò như các slave. Mỗi DataNode chịu trách nhiệm quản lý thông tin lưu trữ của các dữ liệu trên máy mà nó đang chạy. HDFS cung cấp một không gian tên cho phép dữ liệu của người dùng lưu trữ trong các tập tin. Mỗi

tập tin sẽ được tách thành một hoặc nhiều block được lưu trữ trong một tập hợp những DataNode. Dựa vào không gian tên, NameNode có thể thực hiện các thao tác như đóng, mở và đổi tên tập tin và thư mục. NameNode cũng xác định sơ đồ lưu trữ các block của DataNode. Ngoài nhiệm vụ đáp ứng yêu cầu đọc, ghi dữ liệu do Namenode chuyển đến, các DataNode cũng có nhiệm vụ tạo, xóa và nhân rộng các blocks theo những chỉ thị từ NameNode.

File System Namespace

HDFS tổ chức tập tin phân cấp theo mô hình truyền thống. Với HDFS người dùng cũng có thể tạo, xóa, di chuyển, đổi tên tập tin thư mục như các hệ thống tập tin truyền thống thông thường.

Data Replication

HDFS được thiết kế để lưu trữ các tập tin cực lớn. Trên một Hadoop cluster mỗi tập tin được chia thành nhiều block có thứ tự và lưu trữ trên nhiều máy. Việc nhân bản các block nhằm tăng khả năng chịu lỗi cho hệ thống. Mỗi block sẽ được nhân bản bao nhiêu lần tùy theo cấu hình của hệ thống. Các DataNode có nhiệm vụ lưu trữ các block mà nó được NameNode phân công.

Lỗi đĩa, thông điệp HeartBeat và nhân bản lại các block

Định kỳ, mỗi DataNode sẽ gửi đến NameNode một thông điệp gọi là HeartBeats để xác định tình trạng hoạt động của DataNode. Nếu sau một khoảng thời gian quy định mà không thấy DataNode gửi HeartBeats, NameNode sẽ đánh dấu là DataNode đó bị lỗi và không ra bất kỳ thao tác nào cho nó nữa. Bất kỳ dữ liệu gì do DataNode bị lỗi quản lý đều xem như không còn nữa. NameNode sẽ tìm một bản sao khác của các block trên DataNode bị lỗi và sao chép toàn bộ công việc của DataNode bị lỗi cùng bản sao của các block cho DataNode nào còn rảnh để thực hiện tiếp công việc. Khi số lượng bản sao của một block nhỏ hơn giá trị được chỉ định trước, NameNode sẽ khởi tạo quá trình nhân bản bất cứ khi nào có thể.

Đọc dữ liệu trên HDFS

Khi chương trình client gửi một yêu cầu đọc dữ liệu tới HDFS, hệ thống sẽ gửi một yêu cầu đến NameNode để hỏi về vị trí các block có liên quan đến dữ liệu cần đọc. Sau khi có được danh sách vị trí các block có liên quan, Client sẽ kết nối trực tiếp đến DataNode để đọc dữ liệu.

Ghi dữ liệu trên HDFS

Khi chương trình client có tập tin dữ liệu cần ghi lên HDFS, đầu tiên tập tin dữ liệu phải được ghi vào bộ nhớ cục bộ của máy tính chạy chương trình client. Khi tập tin cục bộ đã tích lũy đủ dung lượng của một block, hoặc quá trình ghi tập tin cục bộ đã hoàn toàn kết thúc, chương trình client sẽ nhận về một danh sách những DataNode được NameNode chỉ định sẽ chứa tập tin dữ liệu này. Sau đó, chương trình client sẽ gửi bản sao của tập tin cần ghi đến DataNode đầu tiên. DataNode đầu tiên sẽ trích lấy một phần của tập tin để ghi xuống bộ nhớ của mình, sau đó chuyển toàn bộ phần còn lại cho DataNode khác, DataNode khác sẽ thực hiện lại công việc mà DataNode đầu tiên đã làm, cứ tiếp tục như vậy cho đến khi toàn bộ nội dung của tập tin được ghi hết. Có thể nói, việc ghi dữ liệu trên hệ thống HDFS được thực hiện theo cơ chế ống dẫn.

HDFS là nền tảng của Hadoop và do đó là một thành phần rất quan trọng của hệ sinh thái Hadoop. Đây là phần mềm Java cung cấp nhiều tính năng như khả năng mở rộng, tính sẵn sàng cao, khả năng chịu lỗi, hiệu quả về chi phí, v.v. Nó cũng cung cấp khả năng lưu trữ dữ liệu phân tán mạnh mẽ cho Hadoop. Chúng tôi có thể triển khai nhiều khung phần mềm khác qua HDFS.

3.4 Các thành phần của HDFS

Hadoop hoạt động theo kiểu chủ nô. Có một nút chính và có n số nút phụ trong đó n có thể là 1000. Master quản lý, duy trì và giám sát các nô lệ trong khi các nô lệ là các nút công nhân thực sự. Trong kiến trúc Hadoop, Master nên triển khai trên phần cứng cấu hình tốt, không chỉ phần cứng hàng hóa. Vì nó là trung tâm của cụm Hadoop.

Master lưu trữ siêu dữ liệu (dữ liệu về dữ liệu) trong khi nô lệ là các nút lưu trữ dữ liệu. Các kho lưu trữ dữ liệu phân tán trong cụm. Máy khách kết nối với nút chính để thực hiện bất kỳ tác vụ nào. Bây giờ trong hướng dẫn Hadoop cho người mới bắt đầu này, chúng ta sẽ thảo luận chi tiết về các tính năng khác nhau của Hadoop.

Viết tắt của Hadoop Hệ thống tệp phân tán cung cấp cho lưu trữ phân tán cho Hadoop. HDFS có cấu trúc liên kết chủ-tớ.

Master là một cỗ máy cao cấp, nơi nô lệ là những chiếc máy tính rẻ tiền. Các tệp Dữ liệu lớn được chia thành số khối. Hadoop lưu trữ các khối này theo kiểu phân tán trên cụm các nút nô lệ. Trên trang cái, chúng tôi đã lưu trữ siêu dữ liệu.

HDFS có hai daemon:

NameNode: NameNode thực hiện các chức năng sau:

- + NameNode Daemon chạy trên máy chủ.
- + Chịu trách nhiệm duy trì, giám sát và quản lý các DataNodes.
- + Ghi lại siêu dữ liệu của các tệp như vị trí của các khối, kích thước tệp, quyền, hệ thống phân cấp, v.v.
- + Namenode nắm bắt tất cả các thay đổi đối với siêu dữ liệu như xóa, tạo và đổi tên tệp trong nhật ký chỉnh sửa.
- + Nó thường xuyên nhận được nhịp tim và chặn các báo cáo từ DataNodes.

DataNode: Các chức năng khác nhau của DataNode như sau:

- + DataNode chạy trên máy phụ.
- + Nó lưu trữ dữ liệu kinh doanh thực tế.
- + Nó phục vụ yêu cầu đọc-ghi từ người dùng.
- + DataNode thực hiện công việc cơ bản là tạo, sao chép và xóa các khối bằng lệnh của NameNode.
- + Sau mỗi 3 giây, theo mặc định, nó sẽ gửi nhịp tim đến NameNode để báo cáo tình trạng của HDFS.

Có ba thành phần chính của Hadoop HDFS như sau:

3.4.1 DataNode

Đây là các nút lưu trữ dữ liệu thực tế. HDFS lưu trữ dữ liệu theo cách phân tán. Nó chia các tệp đầu vào có nhiều định dạng khác nhau thành các khối. Các DataNodes lưu trữ từng khối này. Sau đây là các chức năng của DataNodes: -

- Khi khởi động, DataNode bắt tay với NameNode. Nó xác minh ID không gian tên và phiên bản phần mềm của DataNode.
- Ngoài ra, nó gửi một báo cáo khối đến NameNode và xác minh các bản sao khối.
- Nó gửi một nhịp tim đến NameNode sau mỗi 3 giây để báo rằng nó còn sống.

3.4.2. NameNode

NameNode không là gì ngoài nút chính. NameNode chịu trách nhiệm quản

lý không gian tên hệ thống tệp, kiểm soát quyền truy cập tệp của khách hàng. Ngoài ra, nó thực hiện các tác vụ như mở, đóng và đặt tên cho các tệp và thư mục. NameNode có hai tệp chính - FSImage và Nhật ký chỉnh sửa

FSImage - FSImage là ảnh chụp nhanh siêu dữ liệu của HDFS theo thời gian. Nó chứa thông tin như quyền tệp, hạn ngạch đĩa, dấu thời gian sửa đổi, thời gian truy cập, v.v.

Nhật ký chỉnh sửa - Nó chứa các sửa đổi trên FSImage. Nó ghi lại những thay đổi gia tăng như đổi tên tệp, thêm dữ liệu vào tệp, v.v.

Bất cứ khi nào NameNode khởi động, nó sẽ áp dụng nhật ký Chỉnh sửa cho FSImage. Và FSImage mới được tải trên NameNode.

3.4.3 Secondary NameNode

Nếu NameNode không khởi động lại trong nhiều tháng, kích thước của nhật ký Chỉnh sửa sẽ tăng lên. Điều này sẽ làm tăng thời gian chết của cụm khi khởi động lại NameNode. Trong trường hợp này, Mã Tên Phụ xuất hiện trong hình. Mã Tên Phụ áp dụng nhật ký chỉnh sửa trên FSImage theo định kỳ. Và nó cập nhật FSImage mới trên NameNode chính.

Trong HDFS cluster có duy nhất một NameNode, hệ thống không thể hoạt động được nếu như không có NameNode. Vì tính chất quan trọng đó, việc sao lưu dự phòng cho NameNode là rất cần thiết. Đó chính là nhiệm vụ của Secondary NameNode. Định kỳ sau mỗi khoảng thời gian, Secondary NameNode sẽ kết nối đến NameNode để cập nhật các tập tin về cấu hình, trạng thái của hệ thống, toàn bộ các tập tin này sẽ được lưu lại thành một CheckPoint, dùng cho việc khôi phục lại NameNode nếu NameNode bị lỗi. Đồng thời, NameNode là một dịch vụ phải phục vụ rất nhiều DataNode trong hệ thống. Secondary NameNode còn có khả năng chia tải với NameNode. Ngoài ra, việc định kỳ tạo CheckPoint giúp hệ thống chuyển sang trạng thái hoạt động nhanh hơn.

Giám sát hoạt động Hệ thống Hadoop

Ta có thể giám sát tình trạng hoạt động của các MapReduce Job thông qua trình duyệt web, với địa chỉ [http:<JobTracker>:50030](http://<JobTracker>:50030), giá trị <JobTracker> chính là IP hoặc tên miền của máy đang chạy JobTracker, tức máy master của hệ thống. Nhờ đó, ta có thể biết được MapReduce job đã hoàn tất, bị lỗi hay đang chạy, chạy được bao nhiêu phần, node nào đang chạy các TaskTracker, bao nhiêu tác vụ

Reducer, bao nhiêu tác vụ Mapper đang chạy trên các node, số phần trăm kết quả đạt được của các tác vụ, mỗi máy đã xử lý bao nhiêu input split, v.v.

Tương tự với HDFS, ta có thể sử dụng trình duyệt web, truy cập tới địa chỉ `http:<NameNode>:50070`, với `<NameNode>` là địa chỉ IP, hoặc tên miền của máy chạy NameNode, để biết được các thông tin về trạng thái hoạt động của hệ thống.

3.4.4 MapReduce

MapReduce là thành phần xử lý dữ liệu của Hadoop. Nó áp dụng tính toán trên các tập dữ liệu song song do đó cải thiện hiệu suất. MapReduce hoạt động theo hai giai đoạn -

Giai đoạn bản đồ - Giai đoạn này nhận đầu vào là các cặp khóa-giá trị và tạo ra đầu ra dưới dạng các cặp khóa-giá trị. Nó có thể viết logic nghiệp vụ tùy chỉnh trong giai đoạn này. Giai đoạn bản đồ xử lý dữ liệu và cung cấp cho giai đoạn tiếp theo.

Giảm giai đoạn - Khung công tác MapReduce sắp xếp cặp khóa-giá trị trước khi cung cấp dữ liệu cho giai đoạn này. Giai đoạn này áp dụng kiểu tính toán tóm tắt cho các cặp khóa-giá trị.

- Mapper đọc khối dữ liệu và chuyển nó thành các cặp khóa-giá trị.
- Bây giờ, các cặp khóa-giá trị này được nhập vào bộ giảm.
- Bộ giảm tải nhận các bộ dữ liệu từ nhiều trình ánh xạ.
- Bộ giảm áp dụng tổng hợp các bộ giá trị này dựa trên khóa.
- Đầu ra cuối cùng từ bộ giảm tốc được ghi vào HDFS.

Khung công tác MapReduce xử lý sự cố. Nó khôi phục dữ liệu từ một nút khác trong trường hợp một nút gặp sự cố.

3.4.5. Yarn

Yarn là viết tắt của Yet Another Resource Manager. Nó giống như hệ điều hành của Hadoop vì nó giám sát và quản lý các tài nguyên. Yarn xuất hiện trong bức tranh với sự ra mắt của Hadoop 2.x để cho phép các khối lượng công việc khác nhau. Nó xử lý các khối lượng công việc như xử lý luồng, xử lý tương tác, xử lý hàng loạt trên một nền tảng duy nhất. Yarn có hai thành phần chính - Node Manager và Resource Manager.

NodeManager Trình quản lý nút

Nó là tác nhân mỗi nút của Yarn và chăm sóc các nút tính toán riêng lẻ trong một cụm Hadoop. Nó giám sát việc sử dụng tài nguyên như CPU, bộ nhớ, v.v. của nút cục bộ và tương tự như vậy với Trình quản lý tài nguyên.

ResourceManager Quản lý tài nguyên

Nó chịu trách nhiệm theo dõi các tài nguyên trong cụm và lên lịch các tác vụ như công việc thu nhỏ bản đồ.

Ngoài ra, chúng tôi có Trình chủ ứng dụng và Trình lập lịch trong Yarn. Hãy để chúng tôi xem xét chúng.

Ứng dụng Master có hai chức năng và chúng là: -

- Thương lượng tài nguyên từ Người quản lý tài nguyên
- Làm việc với NodeManager để giám sát và thực hiện nhiệm vụ con.

Sau đây là các chức năng của Trình lập lịch tài nguyên: -

- Nó phân bổ tài nguyên cho các ứng dụng đang chạy khác nhau
- Nhưng nó không giám sát trạng thái của ứng dụng. Vì vậy, trong trường hợp thất bại của nhiệm vụ, nó không khởi động lại như cũ.

3.4.6 Hive

Hive là một dự án kho dữ liệu được xây dựng trên Apache Hadoop, cung cấp khả năng truy vấn và phân tích dữ liệu. Nó có ngôn ngữ gọi là HQL hoặc Ngôn ngữ truy vấn Hive. HQL tự động chuyển các truy vấn thành công việc thu nhỏ bản đồ tương ứng.

Các bộ phận chính của Hive là -

- MetaStore - nó lưu trữ siêu dữ liệu
- Trình điều khiển - Quản lý vòng đời của câu lệnh HQL
- Trình biên dịch truy vấn - Biên dịch HQL thành DAG, tức là Đồ thị Acyclic được hướng dẫn
- Máy chủ Hive - Cung cấp giao diện cho máy chủ JDBC / ODBC.

Facebook đã thiết kế Hive cho những người thông thạo SQL. Nó có hai thành phần cơ bản - Dòng lệnh Hive và JDBC, ODBC. Hive Command line là một giao diện để thực thi các lệnh HQL. Và JDBC, ODBC thiết lập kết nối với lưu trữ dữ

liệu. Hive có khả năng mở rộng cao. Nó có thể xử lý cả hai loại khối lượng công việc, tức là xử lý hàng loạt và xử lý tương tác. Nó hỗ trợ kiểu dữ liệu gốc của SQL. Hive cung cấp nhiều chức năng được xác định trước để phân tích. Nhưng bạn cũng có thể xác định các chức năng tùy chỉnh của riêng mình được gọi là UDF hoặc các chức năng do người dùng xác định.

3.4.7 Pig

Pig là một ngôn ngữ giống SQL được sử dụng để truy vấn và phân tích dữ liệu được lưu trữ trong HDFS. Yahoo là người sáng tạo ban đầu của Pig. Nó sử dụng ngôn ngữ latin lộn. Nó tải dữ liệu, áp dụng bộ lọc cho nó và kết xuất dữ liệu ở định dạng cần thiết. Pig cũng bao gồm JVM được gọi là Pig Runtime. Các đặc điểm khác nhau của Pig như sau: -

- Khả năng mở rộng - Để thực hiện xử lý mục đích đặc biệt, người dùng có thể tạo chức năng tùy chỉnh của riêng họ.
- Cơ hội tối ưu hóa - Pig tự động tối ưu hóa truy vấn cho phép người dùng tập trung vào ngữ nghĩa hơn là hiệu quả.
- Xử lý tất cả các loại dữ liệu - Pig phân tích cả có cấu trúc và không có cấu trúc.

3.4.8. Hbase

HBase là một cơ sở dữ liệu NoSQL được xây dựng trên HDFS. Các tính năng khác nhau của HBase là nó là cơ sở dữ liệu phân tán mã nguồn mở, không quan hệ. Nó bắt chước Bigtable của Google và được viết bằng Java. Nó cung cấp quyền truy cập đọc / ghi thời gian thực vào các bộ dữ liệu lớn. Các thành phần khác nhau của nó như sau: -

HBase Master

HBase thực hiện các chức năng sau:

- Duy trì và giám sát cụm Hadoop.
- Thực hiện quản trị cơ sở dữ liệu.
- Kiểm soát chuyển đổi dự phòng.
- HMaster xử lý hoạt động DDL.

RegionServer

Máy chủ vùng là một quá trình xử lý các yêu cầu đọc, ghi, cập nhật và xóa

từ máy khách. Nó chạy trên mọi nút trong một cụm Hadoop là HDFS DataNode.

HBase là một hệ quản trị cơ sở dữ liệu hướng cột. Nó chạy trên HDFS. Nó phù hợp với các tập dữ liệu thừa thớt thường gặp trong các trường hợp sử dụng Dữ liệu lớn. HBase hỗ trợ viết ứng dụng trong Apache Avro, REST và Thrift. Apache HBase có khả năng lưu trữ độ trễ thấp. Doanh nghiệp sử dụng điều này để phân tích thời gian thực.

Thiết kế của HBase là để chứa nhiều bảng. Mỗi bảng này phải có một khóa chính. Các nỗ lực truy cập vào bảng HBase sử dụng khóa chính này.

Ví dụ cho phép chúng ta xem xét bảng HBase lưu trữ nhật ký chân đoán từ máy chủ. Trong trường hợp này, hàng nhật ký điển hình sẽ chứa các cột như dấu thời gian khi nhật ký được ghi. Và máy chủ mà bản ghi bắt nguồn từ đó.

Chương 4 Thực hành Phân tích Dữ liệu lớn Hadoop

4.1 Môi trường Hadoop phục vụ phân tích dữ liệu lớn

4.1.1 Cài đặt phần mềm MS Visual Studio Code (VSC)

MS VCS là môi trường lập trình với các ngôn ngữ phổ biến (Python, Java, C/C++, PHP, JavaScripts/CSS/XML) cho phép kết nối và giao tiếp với môi trường lưu trữ dữ liệu điện toán đám mây trên máy chủ Linux/Windows.

Phần mềm MS VSC có các phiên bản có thể cài đặt trên các Hệ điều hành Windows, Linux, MacOS.

4.1.2. Cài đặt tiện ích SSH Remote Connect Extention trong VSC

Hướng dẫn cài đặt SSH Remote Connect xem trên trang hướng dẫn sử dụng MS VSC tại <https://code.visualstudio.com/learn/develop-cloud/ssh-lab-machines>

4.1.3. Kiểm tra tài khoản lưu trữ dữ liệu cá nhân và thay đổi mật khẩu

Mật khẩu truy cập tài khoản cá nhân (password) trên máy chủ FIT-LAB dùng để kết nối bằng **SSH Connect** từ phần mềm **MS Visual Studio Code**. Lệnh kết nối có thể được khởi động bằng phím **F1** trong **MS VCS**

```
ssh <Mã SV>@fit-lab.vlu.edu.vn -A
```

Trường hợp tên miền fit-lab.vlu.edu.vn không giải được, có thể sử dụng địa chỉ IP bằng lệnh sau:

```
ssh <Mã SV>@ 14.161.47.108 -A
```

4.1.4. Duyệt thư mục lưu trữ dữ liệu cá nhân (Remote Explorer)

Thư mục cá nhân dùng cho thực hành **Hadoop** trong môi trường **MS VSC** sau khi được kết nối thành công với máy chủ **FIT-LAB**, nằm trong thư mục con **iDragonCloud/hadoop** (hoặc **iDragonCloud/MongoDB** sử dụng lưu trữ các dữ liệu thực hành liên quan tới công cụ **MongoDB**)

4.1.5. Truy cập thư mục lưu trữ dữ liệu cá nhân trong điện toán đám mây

Người dùng có thể sử dụng giao diện web khi đăng nhập <https://fit-lab.vlu.edu.vn> (**FIT-LAB**), truy cập menu **Storage** (lưu trữ dữ liệu cá nhân) để xem và quản lý các tập tin và thư mục trong **iDragonCloud**

4.2 Bài toán tính số lần lặp của một từ trong một tập tin văn bản

Đầu vào (dữ liệu tập tin văn bản)

Dữ liệu dùng cho tập tin văn bản (input.txt) chứa các từ tiếng Anh (utf-8)

Đầu ra của tính toán (kết quả phân tích dữ liệu)

Kết quả đầu ra cũng là một tập tin văn bản (output.txt) mà trên mỗi dòng chứa một từ xuất hiện trong tập tin văn bản input.txt cùng số lần xuất hiện của từ đó trong tập tin văn bản đầu vào input.txt

Công cụ tính toán

Công cụ tính toán để giải bài toán trên là môi trường **Hadoop** với giải thuật **MapReduce**, xử lý các dữ liệu luồng (streaming data) bằng ngôn ngữ lập trình Python, giao tiếp qua **STDIN** (luồng nhập dữ liệu chuẩn sys.stdin trong Python) và **STDOUT** (luồng xuất dữ liệu chuẩn sys.stdout trong Python).

4.2.1 Phân tích dữ liệu văn bản <mapper.py>

Bước phân tích sẽ đọc dữ liệu từ đầu vào (STDIN) là các dòng văn bản (line), phân tích để tách các từ (word) và ghi danh sách các từ với chỉ số 1 theo từng dòng vào đầu ra (STDOUT). Bước phân tích mapper sẽ không tính tổng số lần xuất hiện của từ trong tập tin ngay, mà chỉ xuất ra các cặp (word, 1), cho dù từ word này có thể xuất hiện nhiều lần trong tập tin.

Mã nguồn tập tin <mapper.py>

```
#!/usr/bin/env python3

import sys

# input comes from STDIN (standard input)
# remove leading and trailing white space
for line in sys.stdin:
    line = line.strip()

    # split the line into words
    words = line.split()

    # increase counters
    for word in words:
        # write the results to STDOUT (standard output);
        # what we output here will be the input for the
        # Reduce step, i.e. the input for reducer.py
```

```
# tab-delimited; the trivial word count is 1
```

```
print("%s\t%s" %(word, 1))
```

4.2.2 – Tổng hợp dữ liệu văn bản <reducer.py>

Bước tổng hợp này đọc dữ liệu từ đầu vào (STDIN) của <mapper.py> (là các dòng văn bản chứa (word, 1)) và tổng hợp các lần xuất hiện của mỗi từ cho tới tổng số cuối cùng, xuất kết quả ra STDOUT.

Mã nguồn tập tin <**reducer.py**>

```
#!/usr/bin/env python3
```

```
from operator import itemgetter
```

```
import sys
```

```
current_word = None
```

```
current_count = 0
```

```
word = None
```

```
# input comes from STDIN
```

```
for line in sys.stdin:
```

```
    # remove leading and trailing whitespace
```

```
    line = line.strip()
```

```
    # parse the input we got from mapper.py
```

```
    word, count = line.split('\t', 1)
```

```
    # convert count (currently a string) to int
```

```
    try:
```

```
        count = int(count)
```

```
    except ValueError:
```

```
        # count was not a number, so silently
```

```
        # ignore/discard this line
```

```
        continue
```

```
    # this IF-switch only works because Hadoop sorts map output
```

```

# by key (here: word) before it is passed to the reducer

if current_word == word:
    current_count += count
else:
    if current_word:
        # write result to STDOUT
        print("%s\t%s" % (current_word, current_count))
    current_count = count
    current_word = word

# do not forget to output the last word if needed!
# if current_word == word:
#     print '%s\t%s' % (current_word, current_count)

```

4.2.3 Kiểm tra và chạy các mã Python trong mapper.py và reducer.py

```
$ echo "foo foo quux labs foo bar quux" | ./mapper.py
```

```

foo  1
foo  1
quux 1
labs 1
foo  1
bar  1
quux 1

```

```
$ echo "foo foo quux foo bar quux" | ./mapper.py | sort | ./reducer.py
```

```

bar  1
foo  3
labs 1
quux 2

```

4.2.4 Tạo lập dữ liệu HDFS và khởi động Môi trường Hadoop

Format hệ thống tập tin HDFS trong môi trường Hadoop bằng lệnh

```
$ hdfs namenode -format
```

Khởi động Hadoop bằng các lệnh

```
$ start-dfs.sh
```

```
$ start-yarn.sh
```

Tắt hệ thống Hadoop bằng các lệnh

```
$ stop-dfs.sh
```

```
$ stop-yarn.sh
```

Lệnh tổng hợp các tác vụ trên (sau khi đã thực hiện không gặp lỗi)

```
$ start-all.sh
```

```
$ stop-all.sh
```

Kiểm tra hoạt động của môi trường Hadoop bằng lệnh

```
$ jps
```

Kiểm tra (duyệt thư mục và tập tin) hệ thống tập tin HDFS

```
$ hdfs dfs -ls /
```

Tạo thư mục mới trong HDFS

```
$ hdfs dfs -mkdir /In
```

Kiểm tra thư mục vừa tạo

```
$ hdfs dfs -ls /
```

4.2.5 Chép dữ liệu nguồn (tập tin văn bản text) vào hệ thống tập tin HDFS

```
$ hdfs dfs -put ./Data/input.txt /In
```

Kiểm tra tập tin văn bản input.txt đã được lưu trong HDFS

```
$ hdfs dfs -ls /In
```

Chép tập tin từ HDFS về lưu trữ trong thư mục con **Out**

```
$ mkdir ./Out
```

```
$ hdfs dfs -cat /In/input.txt
```

```
$ hdfs dfs -get /In/input.txt ./Out
```

4.2.6 Chạy các lệnh mapper.py và reducer.py trên Hadoop (Map/Reduce)

```
$ mapred streaming -input /In -output ~/Out -mapper ~/mapper.py -reducer  
~/reducer.py
```

Ghi chú: Hệ thống phân tích dữ liệu lớn Hadoop MapReduce sẽ đọc tất cả các tập tin được lưu trong thư mục HDFS ~/In và chạy các đoạn mã tính toán, lưu kết quả trong thư mục ~/Out.

Kết quả phân tích dữ liệu lớn trên Hadoop của thí dụ trên: có thể xem kết xuất của tiến trình phân tích dữ liệu lớn trên Hadoop qua giao diện Web:

<http://fit-lab.vlu.edu.vn:<Personal Hadoop Port>>

4.2.7 Kiểm tra kết quả phân tích dữ liệu trên Hadoop MapReduce

```
$ hdfs dfs -ls /Out
```

Sẽ xuất hiện ít nhất một tập tin kết quả phân tích dữ liệu (**part-00000**)

```
$ hdfs dfs -cat /Out/part-00000
```

4.2.8 Các biến môi trường sử dụng để vận hành Hadoop Cluster

Môi trường Hadoop vận hành dựa trên các cấu hình dành riêng cho từng người sử dụng. Để kiểm tra các biến môi trường này, có thể sử dụng lệnh

```
$ env
```

Biến môi trường vận hành Hadoop sẽ được cấu hình tự động khi người dùng đăng nhập vào máy chủ FIT-LAB và được định nghĩa trước trong tập tin cấu hình ~/.bashrc (thư mục \$HOME của người dùng), hoặc được thiết lập trong tập tin ~/iDragonCloud/hadoop/etc/hadoop/hadoop-env.sh

```
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

```
HADOOP_HOME=$HOME/iDragonCloud/hadoop
```

```
HADOOP_MAPRED_HOME=$HADOOP_HOME
```

```
HADOOP_COMMON_HOME=$HADOOP_HOME
```

```
HADOOP_HDFS_HOME=$HADOOP_HOME
```

```
YARN_HOME=$HADOOP_HOME
```

```
HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
```

PATH=\$PATH:\$HADOOP_HOME/sbin:\$HADOOP_HOME/bin

HADOOP_INSTALL=\$HADOOP_HOME

4.2.9 Các tập tin cấp hình Hadoop

core-site.xml

```
<configuration>

  <property>

    <name>fs.default.name</name>

    <value>hdfs://localhost:<PersonalPort></value>

  </property>

</configuration>
```

hdfs-site.xml

```
<configuration>

  <property>

    <name>dfs.replication</name>

    <value>1</value>

  </property>

  <property>

    <name>dfs.name.dir</name>

    <value>file:///export/users/<UserID>/iDragonCloud/hadoop/hdfs/namenode</value>

  </property>

  <property>

    <name>dfs.data.dir</name>

    <value>file:///export/users/<UserID>/iDragonCloud/hadoop/hdfs/datanode</value>

  </property>

</configuration>
```

yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

mapred-site.xml

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

4.3 Cải tiến mã Python sử dụng cặp (key,value) để chạy với Hadoop Cluster

<mapper.py>

```
#!/usr/bin/env python3

"""A more advanced Mapper, using Python iterators and generators."""

import sys

def read_input(file):
    for line in file:
        # split the line into words
        yield line.split()

def main(separator='\t'):
    # input comes from STDIN (standard input)
    data = read_input(sys.stdin)
    for words in data:
```

```

# write the results to STDOUT (standard output);
# what we output here will be the input for the
# Reduce step, i.e. the input for reducer.py
#
# tab-delimited; the trivial word count is 1
for word in words:
    print("%s%s%d" % (word, separator, 1))
if __name__ == "__main__":
    main()
<reducer.py>
#!/usr/bin/env python3
"""A more advanced Reducer, using Python iterators and generators."""
from itertools import groupby
from operator import itemgetter
import sys

def read_mapper_output(file, separator='\t'):
    for line in file:
        yield line.rstrip().split(separator, 1)

def main(separator='\t'):
    # input comes from STDIN (standard input)
    data = read_mapper_output(sys.stdin, separator=separator)
    # groupby groups multiple word-count pairs by word,
    # and creates an iterator that returns consecutive keys and their group:
    #  current_word - string containing a word (the key)
    #  group - iterator yielding all ["<current_word>", "<count>"]
    items
    for current_word, group in groupby(data, itemgetter(0)):

```



```
try:
    total_count = sum(int(count) for current_word, count in group)
    print("%s%s%d" % (current_word, separator, total_count))
except ValueError:
    # count was not a number, so silently discard this item
    pass

if __name__ == "__main__":
    main()
```

Chương 5 Hướng dẫn viết đồ án môn học

5.1 Bài toán tìm kiếm bạn bè trên Facebook bằng công cụ Hadoop

Facebook là mạng xã hội dựa trên kết nối bạn bè. Một danh sách bạn bè thường là là quan hệ hai chiều trên Facebook: nếu tôi là bạn của bạn, bạn là của tôi). Facebook sử dụng không gian lưu trữ dữ liệu cực lớn và thường phục vụ hàng trăm triệu cho tới hàng tỷ yêu cầu kết bạn mỗi ngày. Khi một người dùng Facebook truy cập **hồ sơ của một thành viên** Facebook khác, nếu được phép sẽ thấy **danh sách bạn bè chung**. Danh sách này thường không thay đổi thường xuyên, vì vậy sẽ rất lãng phí nếu thực hiện tính toán lại mỗi khi người dùng truy cập hồ sơ. Công cụ phân tích dữ liệu lớn Hadoop MapReduce có thể được sử dụng để tính toán và tìm ra bạn bè chung của các thành viên Facebook mỗi ngày một lần và lưu trữ lại kết quả. Sau đó chỉ cần tra cứu nhanh.

Giả sử bạn bè được lưu trữ dưới dạng Người -> [Danh sách bạn bè], danh sách bạn bè của chúng ta sau đó là:

A -> B C D

B -> A C D E

C -> A B D E

D -> A B C E

E -> B C D

Mỗi dòng sẽ là một đối số cho một ánh xạ. Đối với mỗi người bạn trong danh sách bạn bè, trình ánh xạ sẽ xuất ra một cặp khóa-giá trị. Khóa sẽ là một người bạn cùng với người đó. Giá trị sẽ là danh sách bạn bè. Khóa sẽ được sắp xếp sao cho những người bạn theo thứ tự, khiến cho tất cả các cặp bạn bè đi đến cùng một giảm. Điều này rất khó giải thích bằng văn bản, vì vậy hãy làm điều đó và xem liệu bạn có thể nhìn thấy mẫu không. Sau khi tất cả các trình lập bản đồ chạy xong, bạn sẽ có một danh sách như sau:

Đối với bản đồ (A -> B C D):

(A B) -> B C D

(A C) -> B C D

(A D) -> B C D

Đối với bản đồ (B -> A C D E): (Lưu ý rằng A đứng trước B trong khóa)

(A B) -> A C D E

(B C) -> A C D E

(B D) -> A C D E

(B E) -> A C D E

Đối với bản đồ (C -> A B D E):

(A C) -> A B D E

(B C) -> A B D E

(C D) -> A B D E

(C E) -> A B D E

Đối với bản đồ (D -> A B C E):

(A D) -> A B C E

(B D) -> A B C E

(C D) -> A B C E

(D E) -> A B C E

Và cuối cùng cho bản đồ (E -> B C D):

(B E) -> B C D

(C E) -> B C D

(D E) -> B C D

Trước khi chúng tôi gửi các cặp khóa-giá trị này đến Reduce, ta có thể nhóm chúng theo khóa và nhận được:

(A B) -> (A C D E) (B C D)

(A C) -> (A B D E) (B C D)

(A D) -> (A B C E) (B C D)

(B C) -> (A B D E) (A C D E)

(B D) -> (A B C E) (A C D E)

(B E) -> (A C D E) (B C D)

(C D) -> (A B C E) (A B D E)

(C E) -> (A B D E) (B C D)

(D E) -> (A B C E) (B C D)

Mỗi dòng sẽ được chuyển như một đối số cho một bộ Reduce. Hàm giảm sẽ chỉ đơn giản là giao các danh sách các giá trị và xuất cùng một khóa với kết quả của giao. Ví dụ, giảm ((A B) -> (A C D E) (B C D)) sẽ xuất ra (A B): (C D) và có nghĩa là bạn A và B có C và D là bạn chung.

Kết quả sau khi khử là:

(A B) -> (C D)

(A C) -> (B D)

(A D) -> (B C)

(B C) -> (A D E)

(B D) -> (A C E)

(B E) -> (C D)

(C D) -> (A B E)

(C E) -> (B D)

(D E) -> (B C)

Bây giờ khi D truy cập trang cá nhân của B, chúng ta có thể nhanh chóng tra cứu (B D) và thấy rằng họ có ba người bạn chung, (A C E).

5.2 Bài toán phân tích log files từ phần mềm Elearning (Moodle)

Trình bày mô hình phân tích dữ liệu lớn bằng công cụ Hadoop MapReduce

Đây là Chủ đề dành cho các sinh viên chọn Chuyên ngành Công nghệ Dữ liệu

5.3 Các chủ đề ứng dụng phân tích và trực quan hóa dữ liệu bằng MongoDB

Tài chính, thương mại: Phân khúc thị trường và khách hàng, phân tích hành vi khách hàng, phân tích thông tin khách hàng trong thời gian thực, quản lý mối quan hệ khách hàng, phân tích rủi ro, phát hiện gian lận và xếp hạng rủi ro tín dụng...

Dự báo tăng trưởng kinh tế: theo dõi dữ liệu việc làm để nhận biết những lĩnh vực nào đang có lượng tuyển dụng lớn nhất. Đây là ví dụ về một trong những

công cụ mạnh mẽ để đánh giá về thị trường lao động một cách chính xác và thực tế. Dựa vào những dữ liệu thực tế giúp đánh giá mức độ tăng trưởng của một ngành thông qua số lượng thông tin tuyển dụng được đăng tải. Hay như việc xem xét xu hướng tìm kiếm việc làm giúp cho biết nhận thức của người lao động về sức khỏe, tình hình của thị trường lao động và những tác động của nó đối với tăng trưởng kinh tế.

Truyền thông, quảng cáo: phân tích hành vi, thói quen người tiêu dùng, định vị người dùng di động, ghi nhận cuộc gọi trong thời gian thực, tối ưu hoá hệ thống, phân tích xu hướng, hành vi từ việc thu thập lượng thông tin, dữ liệu trực tuyến từ một số lượng lớn các đối tượng sử dụng internet. Đây là một xu hướng đã và đang phát triển mạnh mẽ trong lĩnh vực công nghệ thông tin và cả lĩnh vực kinh tế.

Y tế và chăm sóc sức khỏe: phân tích dữ liệu chuẩn đoán trong thời gian thực, Các nhà cung cấp dịch vụ chăm sóc sức khỏe sẽ sử dụng nguồn dữ liệu thu thập được từ lịch sử bệnh án người bệnh, từ đơn thuốc của bệnh viện, họ sẽ phân tích dữ liệu ngay khi người bệnh vừa được xuất viện để đưa ra các tư vấn về dịch vụ chăm sóc sức khỏe tốt nhất, dự đoán các khả năng biến chứng và thời gian xảy ra biến chứng sau điều trị. Việc phân tích dữ liệu thời gian thực từ các dữ liệu thu thập về hoạt động của người bệnh, tình trạng sức khỏe thời gian thực (do vấn đề tuổi tác, do thể chất từng người) có thể dự đoán được những khả năng phát sinh không mong muốn, do đó sẽ giảm chi phí nằm viện cho người bệnh do có được những cảnh báo từ trước, tiết kiệm cho cả người bệnh và bệnh viện.

Giao thông vận tải: phân tích thời tiết và giao thông trong thời gian thực, tối ưu hoá tuyến đường vận chuyển, giảm thiểu ùn tắc giao thông. Sử dụng số liệu thu thập được tại các tuyến đường: lưu lượng giao thông, khả năng chịu tải phương tiện, số lượng các xe ô tô cá nhân, ô tô lớn, xe hai bánh, xe thô sơ, điều kiện thời tiết, thời gian chờ đèn đỏ, số lượng đèn đỏ trên các tuyến,... Sử dụng những số liệu thu thập được theo thời gian thực và dựa trên số liệu của lịch sử để có những kế hoạch phân luồng giao thông chi tiết, hợp lý, giảm thiểu kẹt xe. Ngoài ra còn đưa ra thông tin cho người tham gia giao thông được biết nếu muốn đi từ nơi này đến nơi khác thì nên đi vào giờ nào để tránh kẹt xe, hoặc đi đường nào là ngắn nhất v.v... Từ đó hình thành một hạ tầng giao thông thông minh

An toàn thông tin, an ninh quốc gia: giám sát an ninh mạng, phát hiện xâm nhập mạng, phân tích tội phạm, phát hiện khủng bố từ các thông tin trên mạng.

Nếu một nhà khai thác nhìn thấy một vụ tắc nghẽn giao thông có thể xảy ra, thì sau đó họ có thể điều chỉnh các tín hiệu giao thông theo đó để giữ luồng xe ô tô chạy thông suốt. Đây là công cụ đặc biệt hữu ích cho các trường hợp khẩn cấp, có thể nói khi một xe cấp cứu đang trên đường đến bệnh viện. Qua thời gian, các thuật toán trong hệ thống sẽ “học” được từ các khuyến nghị thành công nhất, sau đó áp dụng kiến thức khi thực hiện các dự báo tương lai. Việc phân tích các bản ghi dữ liệu sinh ra từ các thiết bị mạng, ứng dụng, gói tin mạng và các sự kiện hệ thống được phục vụ cho mục đích điều tra và phát hiện xâm nhập trong ATTT. Tuy nhiên, các công nghệ truyền thống rất khó khăn trong việc cung cấp các công cụ phân tích dài hạn, quy mô lớn, vì việc lưu trữ số lượng lớn dữ liệu là không khả thi về mặt kinh tế. Kết quả là, hầu hết các bản ghi nhật ký sự kiện trên các hệ thống, thiết bị thường được xóa sau một thời gian duy trì cố định. Sự ra đời của Dữ liệu lớn sẽ chuyển đổi phân tích an toàn thông tin bằng cách thu thập dữ liệu ở một quy mô lớn từ nhiều nguồn, từ các bản ghi nhật ký hệ thống đến các cơ sở dữ liệu về lỗ hổng bảo mật, dữ liệu về tấn công mạng, dữ liệu mã độc... Sau đó sẽ phân tích sâu hơn trên dữ liệu đã có, cung cấp một cái nhìn hợp nhất các thông tin liên quan đến an toàn và đảm bảo được việc phân tích thực hiện theo thời gian thực của dòng dữ liệu.

Hỗ trợ ra quyết định trong các cơ quan chính phủ: các cơ quan nhà nước ngày càng thu thập được khối lượng lớn dữ liệu từ nhiều nguồn như điều tra thống kê, tiếp nhận, xử lý dịch vụ trực tuyến, các dữ liệu này theo thời gian sẽ trở thành kho dữ liệu lớn của các cơ quan. Việc khai thác sử dụng dữ liệu lớn trong cơ quan nhà nước sẽ ngày càng có vai trò quan trọng phục vụ cho hoạt động của cơ quan nhà nước.

Tài liệu tham khảo

- 1. Handbook of Big Data Technologies**, Albert Y. Zomaya, Sherif Sakr, Springer International Publishing AG 2017
- 2. Introducing Data Science - Big Data, Machine Learning and More, Using Python Tools.** - Davy Cielen, Arno D. B. Meysman, Mohamed Ali - ©2016 by Manning Publications Co.
- 3. BigData and Analytics** - Vincenzo Morabito – Springer International Publishing Switzerland 2015
- 4. MongoDB Architecture Guide: Overview** - July 2020
- 5. Hadoop Tutorials – Tutorials Point** - @2014
- 5. Hadoop with Python**, Zachary Radtka & Donald Miner - © 2016 by O'Reilly Media, Inc.